INFORME INTEGRADOR

BODEGAS DEL ATUEL

Fases 1 a 8 · Informe acumulativo completo

Trabajo Práctico Integrador de Bases de Datos

Ingeniería en Sistemas de Información · UTN FRSR

Alcance de esta compilación

Este documento reúne las entregas de las ocho fases del trabajo integrador. Se incorpora íntegra la entrega original de Fase 1 (entidades, cardinalidades, esquema relacional, normalización y diccionario de datos), junto a los informes ya integrados de Fases 2 a 7.

La Fase 8 presenta el despliegue en la nube, las pruebas funcionales y los controles del respaldo principal. La documentación requerida para Fase 9 se completa en este mismo informe, en el README y en la matriz de trazabilidad, sin generar un informe independiente.

Cierre documental · Noviembre de 2026

CONTENIDO

Sección	Ubicación
Fase 1 · Modelado y normalización	Páginas 3–12 · Entrega original completa
Fase 2 · Integridad y concurrencia	Páginas 13–18 · Informe original
Trazabilidad RN-01 a RN-15 (Fase 2)	Página 19 · Anexo original
Fase 3 · Carga masiva	Páginas 20–25 · Informe original
Fase 4 · Índices y rendimiento	Páginas 26–41 · Informe original
Fase 5 · Consultas SQL	Página 42 · Síntesis y fuentes
Fase 6 · API REST y pruebas	Páginas 43–44 · Síntesis y fuentes
Fase 7 · Frontend e integración	Páginas 45–46 · Síntesis y evidencias
Fase 8 · Nube, seguridad, pruebas y backup	Páginas 47–49
Cierre documental de Fase 9	Página 50

Criterio editorial

Se conserva el texto y el diseño de las entregas originales. La Fase 1 se incorpora completa, con sus tablas y diccionario de datos, en lugar de la síntesis provisional. Las referencias históricas a un informe «parcial» en las fases anteriores corresponden a sus versiones de trabajo preservadas.

Fuentes

Consigna oficial TPI Bodegas del Atuel; entrega original Fase 1.pdf aportada por el equipo; informes de Fases 2 a 7; repositorio videlalauti/trabajo_integrador_bd, rama integracion/fase8 y PR #23 fusionado; actas de Fase 8.

Entidad, atributos y relaciones

Entidad	Atributos
LOCALIDAD	id_localidad, cod_postal, nombre, provincia, encargado_regional
SALA_VENTA	id_sala, dirección ▸ calle, número, dia_cierre
EMPLEADO	cuit, nombre, fecha_ingreso, salario, */antigüedad*
FINCA	id_finca, nombre, superficie_ha, ubicación ▸ departamento, provincia
PARCELA	nro_parcela (discr.), superficie_ha, anio_plantacion
CEPA	id_cepa, nombre, tipo (tinta/blanca)
LOTE_UVA	nro_lote, campania, fecha_cosecha, kg_cosechados
ANALISIS_CALIDAD	id_analisis, fecha, grado_brix, ph, acidez_total_gl
BODEGA	cuit, razon_social, domicilio
VINO	cod_vino, nombre, tipo, anada, volumen_ml, graduacion, precio
MARIDAJE	id_maridaje, nombre, temperatura_servicio_c
CLIENTE	id_cliente, nombre_razon_social, tipo_doc, nro_doc, dirección, fecha_alta, descuento
SOCIO	(sin atributos propios)
DISTRIBUIDOR	zona_cobertura, nro_habilitacion
EXPORTADOR	pais_destino
PERMISO_EXPORTACION	codigo_permiso, fecha_emision, fecha_vencimiento
COMPROBANTE	nro_comprobante, fecha, */subtotal*, */descuento*, */total*
TRANSPORTISTA	cod_transporte, razon_social, cuit, email, telefono, domicilio
INGRESO	cod_ingreso, fecha, estado
TRANSFERENCIA	nro_solicitud, fecha_confeccion, estado
STOCK_DEPOSITO	cantidad
AUDITORIA_PRECIO	id_auditoria, precio_anterior, precio_nuevo, fecha_hora, usuario

Relaciones y Cardinalidad (min, max)

Relación	Entidad A	Entidad B	Tipo	Atributos de la relación
UBICADA_EN	SALA_VENTA (1,1)	LOCALIDAD (0,N)	1:N	—
TRABAJA_EN	EMPLEADO (1,1)	SALA_VENTA (1,N)	1:N	—
SE_DIVIDE_EN (ident.)	FINCA (1,N)	PARCELA (1,1)	1:N	—
PLANTADA_CON	PARCELA (1,1)	CEPA (0,N)	1:N	—
SE_COSECHA	LOTE_UVA (1,1)	PARCELA (0,N)	1:N	—
SE_ENTREGA_A	LOTE_UVA (1,1)	BODEGA (0,N)	1:N	—
SE_ANALIZA	ANALISIS_CALIDAD (1,1)	LOTE_UVA (1,N)	1:N	—
ELABORA	VINO (1,1)	BODEGA (1,N)	1:N	—
COMPUESTO_POR	VINO (1,N)	CEPA (0,N)	N:M	porcentaje
RECOMIENDA	VINO (1,N)	MARIDAJE (0,N)	N:M	—
RESIDE_EN	CLIENTE (1,1)	LOCALIDAD (0,N)	1:N	—
TIENE_PERMISO	EXPORTADOR (0,N)	PERMISO_ EXPORTACION (1,1)	1:N	—
COMPRA	COMPROBANTE (1,1)	CLIENTE (0,N)	1:N	—
SE_EMITE_EN	COMPROBANTE (1,1)	SALA_VENTA (0,N)	1:N	—
CONTIENE	COMPROBANTE (1,N)	VINO (0,N)	N:M	cantidad, precio_unitario, descuento, /subtotal
TIENE_STOCK_SALA	SALA_VENTA (0,N)	VINO (0,N)	N:M	cantidad
REGISTRA_STOCK (ident.)	VINO (0,1)	STOCK_DEPOSITO (1,1)	1:1	—
TRASLADA	INGRESO (1,1)	TRANSPORTISTA (0,N)	1:N	—
PROVIENE_DE	INGRESO (1,1)	BODEGA (0,N)	1:N	—
INCLUYE_ING	INGRESO (0,N)	VINO (0,N)	N:M	cantidad
SOLICITA	TRANSFERENCIA (1,1)	SALA_VENTA (0,N)	1:N	—
INCLUYE_TRF	TRANSFERENCIA (0,N)	VINO (0,N)	N:M	cantidad
CUBRE	TRANSPORTISTA (0,N)	LOCALIDAD (0,N)	N:M	—
AUDITA	AUDITORIA_PRECIO (1,1)	VINO (0,N)	1:N	—
ES_UN	CLIENTE	SOCIO / DISTRIBUIDOR / EXPORTADOR	Esp. total, disjunta	—

Diagrama Entidad Relación

Link de Miro donde fue desarrollado el Diagrama: https://miro.com/app/board/uXjVHMw--T0=/?share_link_id=677013980453

Esquema Relacional

1. Localidades, salas de venta y empleados

LOCALIDAD(id_localidad, cod_postal, nombre, provincia, encargado_regional)

PK: id_localidad

SALA_VENTA(id_sala, calle, numero, dia_cierre, id_localidad)

PK: id_sala

FK: id_localidad → LOCALIDAD(id_localidad)

EMPLEADO(cuit, nombre, fecha_ingreso, salario, id_sala)

PK: cuit

FK: id_sala → SALA_VENTA(id_sala)

2. Producción de uva

FINCA(id_finca, nombre, superficie_ha, departamento, provincia)

PK: id_finca

CEPA(id_cepa, nombre, tipo)

PK: id_cepa

PARCELA(id_finca, nro_parcela, superficie_ha, anio_plantacion, id_cepa)

PK compuesta: (id_finca, nro_parcela)

FK: id_finca → FINCA(id_finca)

FK: id_cepa → CEPA(id_cepa)

BODEGA(cuit, razon_social, domicilio)

PK: cuit

LOTE_UVA(nro_lote, campania, fecha_cosecha, kg_cosechados, id_finca, nro_parcela, cuit_bodega)

PK: nro_lote

FK compuesta: (id_finca, nro_parcela) → PARCELA(id_finca, nro_parcela)

FK: cuit_bodega → BODEGA(cuit)

UNIQUE: (id_finca, nro_parcela, campania)

ANALISIS_CALIDAD(id_analisis, fecha, grado_brix, ph, acidez_total_gl, nro_lote)

PK: id_analisis

FK: nro_lote → LOTE_UVA(nro_lote)

3. Vinos y composición

VINO(cod_vino, nombre, tipo, anada, volumen_ml, graduacion, precio, cuit_bodega)

PK: cod_vino

FK: cuit_bodega → BODEGA(cuit)

VINO_CEPA(cod_vino, id_cepa, porcentaje)

PK compuesta: (cod_vino, id_cepa)

FK: cod_vino → VINO(cod_vino)

FK: id_cepa → CEPA(id_cepa)

MARIDAJE(id_maridaje, nombre, temperatura_servicio_c)

PK: id_maridaje

VINO_MARIDAJE(cod_vino, id_maridaje)

PK compuesta: (cod_vino, id_maridaje)

FK: cod_vino → VINO(cod_vino)

FK: id_maridaje → MARIDAJE(id_maridaje)

4. Clientes y permisos de exportación

TIPO_CLIENTE(tipo_cliente, descuento)

PK: tipo_cliente

CLIENTE(id_cliente, nombre_razon_social, tipo_doc, nro_doc, direccion, fecha_alta, id_localidad, tipo_cliente, descuento_especial, zona_cobertura, nro_habilitacion, pais_destino)

PK: id_cliente

FK: id_localidad → LOCALIDAD(id_localidad)

FK: tipo_cliente → TIPO_CLIENTE(tipo_cliente)

PERMISO_EXPORTACION(codigo_permiso, id_cliente, fecha_emision, fecha_vencimiento)

PK: codigo_permiso

FK: id_cliente → CLIENTE(id_cliente)

5. Ventas y comprobantes

COMPROBANTE(nro_comprobante, fecha, id_cliente, id_sala, subtotal, total, descuento)

PK: nro_comprobante

FK: id_cliente → CLIENTE(id_cliente)

FK: id_sala → SALA_VENTA(id_sala)

RENGLON_COMPROBANTE(nro_comprobante, cod_vino, cantidad, precio_unitario, descuento, subtotal)

PK compuesta: (nro_comprobante, cod_vino)

FK: nro_comprobante → COMPROBANTE(nro_comprobante)

FK: cod_vino → VINO(cod_vino)

6. Transportistas y cobertura

TRANSPORTISTA(cod_transporte, razon_social, cuit, email, telefono, domicilio)

PK: cod_transporte

TRANSPORTISTA_LOCALIDAD(cod_transporte, id_localidad)

PK compuesta: (cod_transporte, id_localidad)

FK: cod_transporte → TRANSPORTISTA(cod_transporte)

FK: id_localidad → LOCALIDAD(id_localidad)

7. Ingresos de mercadería

INGRESO(cod_ingreso, fecha, estado, cod_transporte, cuit_bodega)

PK: cod_ingreso

FK: cod_transporte → TRANSPORTISTA(cod_transporte)

FK: cuit_bodega → BODEGA(cuit)

RENGLON_INGRESO(cod_ingreso, cod_vino, cantidad)

PK compuesta: (cod_ingreso, cod_vino)

FK: cod_ingreso → INGRESO(cod_ingreso)

FK: cod_vino → VINO(cod_vino)

8. Transferencias de mercadería

TRANSFERENCIA(nro_solicitud, fecha_confeccion, estado, id_sala)

PK: nro_solicitud

FK: id_sala → SALA_VENTA(id_sala)

RENGLON_TRANSFERENCIA(nro_solicitud, cod_vino, cantidad)

PK compuesta: (nro_solicitud, cod_vino)

FK: nro_solicitud → TRANSFERENCIA(nro_solicitud)

FK: cod_vino → VINO(cod_vino)

9. Control de stock

STOCK_SALA(id_sala, cod_vino, cantidad)

PK compuesta: (id_sala, cod_vino)

FK: id_sala → SALA_VENTA(id_sala)

FK: cod_vino → VINO(cod_vino)

STOCK_DEPOSITO(cod_vino, cantidad)

PK: cod_vino

FK: cod_vino → VINO(cod_vino)

10. Auditoría

AUDITORIA_PRECIO(id_auditoria, cod_vino, precio_anterior, precio_nuevo, fecha_hora, usuario)

PK: id_auditoria

FK: cod_vino → VINO(cod_vino)

Revisamos el esquema relacional que obtuvimos a partir del DER para comprobar que sus tablas estén normalizadas hasta tercera forma normal. Tomamos como referencia las claves primarias, las claves foráneas y las reglas de negocio del enunciado. Buscamos evitar datos repetidos que puedan provocar inconsistencias al insertar, actualizar o eliminar registros.

Justificación del mapeo de la jerarquía de clientes

Para representar la jerarquía de clientes en el esquema relacional, decidimos utilizar una **tabla única (CLIENTE)**, que reúne los datos generales y los atributos específicos de los tres tipos de clientes: socios, distribuidores y exportadores.

La tabla incluye el atributo tipo_cliente, que permite identificar a qué categoría pertenece cada cliente. Este atributo funciona como clave foránea hacia TIPO_CLIENTE, donde se almacenan los descuentos generales de cada categoría. Además, incorporamos descuento_especial para contemplar posibles excepciones individuales.

Elegimos esta estrategia porque simplifica el diseño y evita tener que crear una tabla adicional para cada subtipo. Además, facilita las consultas, ya que la información de los clientes se encuentra en una misma tabla y no es necesario unir varias tablas para obtener sus datos.

Como desventaja, algunos atributos pueden quedar vacíos dependiendo del tipo de cliente. Ejemplo, un socio no necesita tener zona de cobertura, número de habilitación ni país de destino. Para evitar inconsistencias, estas condiciones se controlarán con restricciones en la base de datos.

La jerarquía se considera **total y disjunta**, ya que todos los clientes deben pertenecer a uno de los tres tipos y ninguno puede pertenecer a más de uno al mismo tiempo.

Consideramos que esta alternativa es adecuada para el sistema porque permite representar los tres tipos de clientes de manera sencilla, manteniendo sus características particulares y facilitando las futuras operaciones sobre la base de datos.

Dependencias funcionales relevantes

Una dependencia funcional indica que el valor de un atributo, o de un conjunto de atributos, permite determinar otro. En nuestro esquema destacamos las siguientes

Relación	Dependencia funcional
LOCALIDAD	id_localidad → nombre, provincia, cod_postal, encargado_regional
SALA_VENTA	id_sala → calle, numero, dia_cierre, id_localidad
PARCELA	(id_finca, nro_parcela) → superficie_ha, anio_plantacion, id_cepa
VINO_CEPA	(cod_vino, id_cepa) → porcentaje
STOCK_SALA	(id_sala, cod_vino) → cantidad
RENGLON_COMPROBANTE	(nro_comprobante, cod_vino) → cantidad, precio_unitario, descuento
TIPO_CLIENTE (nueva)	tipo_cliente → descuento
CLIENTE (corregido)	id_cliente → tipo_cliente, descuento_especial y demás datos del cliente

Verificación de las formas normales

Primera Forma Normal (1FN). Los atributos se representan mediante valores simples. Por ejemplo, la dirección de SALA_VENTA se separa en calle y número. Las relaciones de muchos a muchos, como la composición vino-cepa o la cobertura de transportistas, se resolvieron en tablas intermedias. No dejamos listas de valores dentro de una misma columna.

Segunda Forma Normal (2FN). Las tablas con claves simples no pueden presentar dependencias respecto de una parte de su clave. En las que tienen claves compuestas comprobamos que los datos propios dependen de la combinación completa: el porcentaje de VINO_CEPA depende del vino y de la cepa; la cantidad de STOCK_SALA depende de la sala y del vino. No identificamos dependencias parciales en esas relaciones.

Tercera Forma Normal (3FN). Verificamos que los datos de una entidad no se repitan en otras por medio de dependencias transitivas. Por ejemplo, el encargado regional se conserva en LOCALIDAD, en lugar de repetirse en cada SALA_VENTA. La principal dependencia transitiva que encontramos estaba en los descuentos de CLIENTE, y la separamos como se describe a continuación

Modificación realizada: descuentos de clientes

Antes, CLIENTE contenía tanto tipo_cliente como descuento. Dado que el descuento general depende del tipo de cliente, aparecía la dependencia id_cliente → tipo_cliente → descuento. Esto obligaba a repetir el mismo porcentaje en todos los clientes de un tipo y podía producir diferencias entre registros.

Para corregirlo incorporamos TIPO_CLIENTE(tipo_cliente, descuento), con una fila por tipo y su descuento general (socio 10 %, distribuidor 20 % y exportador 30 %). En CLIENTE eliminamos descuento, mantenemos tipo_cliente como clave foránea y agregamos descuento_especial, que es opcional y representa una excepción propia de un cliente, no un valor determinado por su tipo.

Dejamos para la etapa de consultas SQL la aplicación concreta de U6. Esta decisión no cambia la justificación de la separación de descuentos generales y excepciones individuales.

Otras decisiones que conservamos

Mantenemos PARCELA identificada por (id_finca, nro_parcela), porque su número solo tiene significado dentro de una finca. También conservamos la jerarquía de clientes mediante una única tabla CLIENTE con un tipo y campos específicos opcionales: es una estrategia de implementación que requerirá restricciones para impedir combinaciones inválidas.

En las ventas conservamos el precio unitario y el descuento aplicados en cada renglón porque necesitamos conocer las condiciones de la operación en ese momento. Los subtotales y totales deberán calcularse o mantenerse de forma consistente mediante la lógica de la base de datos, tal como exige el trabajo.

Diccionario de Datos

Tabla	Columna	Tipo	NULL	Restricciones	Descripción
LOCALIDAD	id_localidad	INTEGER	No	PK	Identificador de la localidad.
LOCALIDAD	cod_postal	VARCHAR(12)	No		Código postal de la localidad.
LOCALIDAD	nombre	VARCHAR(100)	No		Nombre de la localidad.
LOCALIDAD	provincia	VARCHAR(100)	No		Provincia correspondiente
LOCALIDAD	encargado_regional	VARCHAR(120)	No		Responsable comercial regional.
SALA_VENTA	id_sala	INTEGER	No	PK	Identificador de la sala.
SALA_VENTA	calle	VARCHAR(120)	No		Calle de la dirección.
SALA_VENTA	numero	VARCHAR(15)	No		Número de la dirección.
SALA_VENTA	dia_cierre	VARCHAR(12)	No	CHECK día válido	Día de cierre semanal.
SALA_VENTA	id_localidad	INTEGER	No	FK → LOCALIDAD(id_localidad)	Localidad donde se ubica la sala.
EMPLEADO	cuit	CHAR(11)	No	PK; DOMAIN CUIT	CUIT del empleado.
EMPLEADO	nombre	VARCHAR(120)	No		Nombre completo del empleado.
EMPLEADO	fecha_ingreso	DATE	No		Fecha de ingreso laboral.
EMPLEADO	salario	NUMERIC(10,2)	No	CHECK > 0	Remuneración actual.
EMPLEADO	id_sala	INTEGER	No	FK → SALA_VENTA(id_sala)	Sala en la que trabaja.
FINCA	id_finca	INTEGER	No	PK	Identificador de finca.
FINCA	nombre	VARCHAR(120)	No		Nombre de la finca.
FINCA	superficie_ha	NUMERIC(10,2)	No	CHECK > 0	Superficie total en hectáreas.
FINCA	departamento	VARCHAR(100)	No		Departamento de ubicación.
FINCA	provincia	VARCHAR(100)	No		Provincia de ubicación.
CEPA	id_cepa	INTEGER	No	PK	Identificador de la cepa.
CEPA	nombre	VARCHAR(100)	No		Nombre de la cepa.
CEPA	tipo	VARCHAR(10)	No	CHECK / ENUM	Clasificación tinta o blanca.
PARCELA	id_finca	INTEGER	No	PK compuesta; FK → FINCA(id_finca)	Finca a la que pertenece.
PARCELA	nro_parcela	INTEGER	No	PK compuesta; CHECK > 0	Número de parcela dentro de la finca.
PARCELA	superficie_ha	NUMERIC(10,2)	No	CHECK > 0	Superficie de la parcela.
PARCELA	anio_plantacion	INTEGER	No		Año en que se plantó.
PARCELA	id_cepa	INTEGER	No	FK → CEPA(id_cepa)	Única cepa plantada.
BODEGA	cuit	CHAR(11)	No	PK; DOMAIN CUIT	CUIT identificador de la bodega.
BODEGA	razon_social	VARCHAR(150)	No		Razón social.
BODEGA	domicilio	VARCHAR(200)	No		Domicilio registrado.
LOTE_UVA	nro_lote	INTEGER	No	PK	Identificador del lote de cosecha.
LOTE_UVA	campania	INTEGER	No		Año/campaña de cosecha.
LOTE_UVA	fecha_cosecha	DATE	No		Fecha de la cosecha.
LOTE_UVA	kg_cosechados	NUMERIC(14,2)	No	CHECK > 0	Peso cosechado en kilogramos.
LOTE_UVA	id_finca	INTEGER	No	FK → PARCELA(id_finca, nro_parcela)	Finca de origen de la parcela.

LOTE_UVA	nro_parcela	INTEGER	No	FK → PARCELA(id_finca, nro_parcela)	Parcela cosechada de la finca.
LOTE_UVA	cuit_bodega	CHAR(11)	No	FK → BODEGA(cuit)	Bodega a la que se entregó la uva.
ANALISIS_CALIDAD	id_analisis	INTEGER	No	PK	Identificador del análisis.
ANALISIS_CALIDAD	fecha	DATE	No		Fecha del análisis.
ANALISIS_CALIDAD	grado_brix	NUMERIC(5,2)	No		Concentración de azúcar expresada en grados Brix.
ANALISIS_CALIDAD	ph	NUMERIC(4,2)	No		Medida de pH.
ANALISIS_CALIDAD	acidez_total_gl	NUMERIC(6,2)	No		Acidez total en gramos por litro.
ANALISIS_CALIDAD	nro_lote	INTEGER	No	FK → LOTE_UVA(nro_lote)	Lote al que se realiza el análisis.
VINO	cod_vino	INTEGER	No	PK	Código de identificación del vino.
VINO	nombre	VARCHAR(150)	No		Nombre comercial del vino.
VINO	tipo	VARCHAR(15)	No	CHECK / ENUM	Tinto, blanco, rosado o espumante.
VINO	anada	INTEGER	No		Año de añada.
VINO	volumen_ml	INTEGER	No	CHECK > 0	Volumen de presentación en mililitros.
VINO	graduacion	NUMERIC(4,2)	No	CHECK 5 a 20	Graduación alcohólica.
VINO	precio	NUMERIC(14,2)	No	CHECK > 0	Precio vigente para todas las salas.
VINO	cuit_bodega	CHAR(11)	No	FK → BODEGA(cuit)	Bodega elaboradora.
VINO_CEPA	cod_vino	INTEGER	No	PK compuesta; FK → VINO(cod_vino)	Vino del corte.
VINO_CEPA	id_cepa	INTEGER	No	PK compuesta; FK → CEPA(id_cepa)	Cepa utilizada.
VINO_CEPA	porcentaje	NUMERIC(5,2)	No	CHECK > 0 y <= 100	Porcentaje de la cepa en el vino.
MARIDAJE	id_maridaje	INTEGER	No	PK	Identificador del maridaje.
MARIDAJE	nombre	VARCHAR(120)	No		Alimento o preparación recomendada.
MARIDAJE	temperatura_servicio_c	NUMERIC(4,1)	No		Temperatura de servicio sugerida en °C.
VINO_MARIDAJE	cod_vino	INTEGER	No	PK compuesta; FK → VINO(cod_vino)	Vino recomendado.
VINO_MARIDAJE	id_maridaje	INTEGER	No	PK compuesta; FK → MARIDAJE(id_maridaje)	Maridaje asociado.
TIPO_CLIENTE	tipo_cliente	VARCHAR(20)	No	PK	Código del tipo: socio, distribuidor o exportador.
TIPO_CLIENTE	descuento	NUMERIC(5,2)	No	CHECK 0 a 100	Porcentaje de descuento general.
CLIENTE	id_cliente	INTEGER	No	PK	Identificador del cliente.
CLIENTE	nombre_razon_social	VARCHAR(150)	No		Nombre completo o razón social.
CLIENTE	tipo_doc	VARCHAR(15)	No		Tipo de documento.
CLIENTE	nro_doc	VARCHAR(30)	No		Número del documento.
CLIENTE	direccion	VARCHAR(200)	No		Domicilio del cliente.
CLIENTE	fecha_alta	DATE	No	DEFAULT CURRENT_DATE	Fecha de alta en el sistema.
CLIENTE	id_localidad	INTEGER	No	FK → LOCALIDAD(id_localidad)	Localidad de residencia.
CLIENTE	tipo_cliente	VARCHAR(20)	No	FK → TIPO_CLIENTE(tipo_cliente)	Clasificación del cliente.
CLIENTE	descuento_especial	NUMERIC(5,2)	Sí	CHECK 0 a 100	Descuento excepcional individual si corresponde.
CLIENTE	zona_cobertura	VARCHAR(150)	Sí		Zona de cobertura de distribuidores.
CLIENTE	nro_habilitacion	VARCHAR(50)	Sí		Habilitación de distribuidores.
CLIENTE	pais_destino	VARCHAR(100)	Sí		Destino principal de exportadores.
PERMISO_EXPORTACION	codigo_permiso	VARCHAR(60)	No	PK	Código identificador del permiso.
PERMISO_EXPORTACION	id_cliente	INTEGER	No	FK → CLIENTE(id_cliente)	Cliente exportador titular del permiso.
PERMISO_EXPORTACION	fecha_emision	DATE	No		Fecha de emisión.
PERMISO_EXPORTACION	fecha_vencimiento	DATE	No	CHECK > fecha_emision	Fecha de vencimiento.
COMPROBANTE	nro_comprobante	INTEGER	No	PK	Número identificador del comprobante.
COMPROBANTE	fecha	TIMESTAMP	No	DEFAULT CURRENT_TIMESTAMP	Fecha y hora de la venta.
COMPROBANTE	id_cliente	INTEGER	No	FK → CLIENTE(id_cliente)	Cliente que realiza la compra.
COMPROBANTE	id_sala	INTEGER	No	FK → SALA_VENTA(id_sala)	Sala en la que se vende.
COMPROBANTE	subtotal	NUMERIC(14,2)	No	DEFAULT 0; CHECK >= 0; trigger	Importe antes de aplicar el descuento; se actualiza desde los renglones.
COMPROBANTE	descuento	NUMERIC(14,2)	No	DEFAULT 0; CHECK >= 0; trigger	Importe total descontado en la venta, en unidades monetarias.
COMPROBANTE	total	NUMERIC(14,2)	No	DEFAULT 0; CHECK >= 0; trigger	Importe final a pagar, calculado como subtotal menos descuento.
RENGLON_COMPROBANTE	nro_comprobante	INTEGER	No	PK compuesta; FK → COMPROBANTE(nro_comprobante)	Comprobante al que pertenece el renglón.

RENGLON_COMPROBANTE	cod_vino	INTEGER	No	PK compuesta; FK → VINO(cod_vino)	Vino vendido.
RENGLON_COMPROBANTE	cantidad	INTEGER	No	CHECK > 0	Cantidad de botellas.
RENGLON_COMPROBANTE	precio_unitario	NUMERIC(14,2)	No	CHECK > 0	Precio histórico al momento de la venta.
RENGLON_COMPROBANTE	descuento	NUMERIC(5,2)	No	CHECK 0 a 100	Porcentaje aplicado al renglón al vender.
RENGLON_COMPROBANTE	subtotal	NUMERIC(14,2)	No	CHECK >= 0; trigger	Importe neto del renglón: cantidad × precio unitario × (1 − descuento / 100).
TRANSPORTISTA	cod_transporte	INTEGER	No	PK	Código del transportista.
TRANSPORTISTA	razon_social	VARCHAR(150)	No		Razón social.
TRANSPORTISTA	cuit	CHAR(11)	No	DOMAIN CUIT; UNIQUE	CUIT de la empresa.
TRANSPORTISTA	email	VARCHAR(150)	No		Correo de contacto.
TRANSPORTISTA	telefono	VARCHAR(30)	No		Número telefónico.
TRANSPORTISTA	domicilio	VARCHAR(200)	No		Domicilio de la empresa.
TRANSPORTISTA_LOCALIDAD	cod_transporte	INTEGER	No	PK compuesta; FK → TRANSPORTISTA(cod_transporte)	Transportista que ofrece cobertura.
TRANSPORTISTA_LOCALIDAD	id_localidad	INTEGER	No	PK compuesta; FK → LOCALIDAD(id_localidad)	Localidad alcanzada.
INGRESO	cod_ingreso	INTEGER	No	PK	Identificador del ingreso al depósito.
INGRESO	fecha	DATE	No		Fecha de ingreso.
INGRESO	estado	estado_documento	No	ENUM; DEFAULT en_confeccion	Estado del ingreso.
INGRESO	cod_transporte	INTEGER	No	FK → TRANSPORTISTA(cod_transporte)	Transportista encargado.
INGRESO	cuit_bodega	CHAR(11)	No	FK → BODEGA(cuit)	Bodega de origen.
RENGLON_INGRESO	cod_ingreso	INTEGER	No	PK compuesta; FK → INGRESO(cod_ingreso)	Ingreso al que pertenece el renglón.
RENGLON_INGRESO	cod_vino	INTEGER	No	PK compuesta; FK → VINO(cod_vino)	Vino recibido.
RENGLON_INGRESO	cantidad	INTEGER	No	CHECK > 0	Botellas ingresadas.
TRANSFERENCIA	nro_solicitud	INTEGER	No	PK	Número de la transferencia.
TRANSFERENCIA	fecha_confeccion	DATE	No	DEFAULT CURRENT_DATE; trigger	Fecha actual de creación; no modificable.
TRANSFERENCIA	estado	estado_documento	No	ENUM; DEFAULT en_confeccion	Estado de transferencia.
TRANSFERENCIA	id_sala	INTEGER	No	FK → SALA_VENTA(id_sala)	Sala solicitante.
RENGLON_TRANSFERENCIA	nro_solicitud	INTEGER	No	PK compuesta; FK → TRANSFERENCIA(nro_solicitud)	Transferencia correspondiente.
RENGLON_TRANSFERENCIA	cod_vino	INTEGER	No	PK compuesta; FK → VINO(cod_vino)	Vino solicitado.
RENGLON_TRANSFERENCIA	cantidad	INTEGER	No	CHECK > 0	Cantidad de botellas solicitadas.
STOCK_SALA	id_sala	INTEGER	No	PK compuesta; FK → SALA_VENTA(id_sala)	Sala de venta.
STOCK_SALA	cod_vino	INTEGER	No	PK compuesta; FK → VINO(cod_vino)	Vino disponible.
STOCK_SALA	cantidad	INTEGER	No	CHECK >= 0	Botellas disponibles.
STOCK_DEPOSITO	cod_vino	INTEGER	No	PK; FK → VINO(cod_vino)	Vino almacenado en el depósito central.
STOCK_DEPOSITO	cantidad	INTEGER	No	CHECK >= 0	Botellas existentes en depósito.
AUDITORIA_PRECIO	id_auditoria	INTEGER	No	PK	Identificador del cambio de precio
AUDITORIA_PRECIO	cod_vino	INTEGER	No	FK → VINO(cod_vino)	Vino cuyo precio cambió.
AUDITORIA_PRECIO	precio_anterior	NUMERIC(14,2)	No		Precio anterior al cambio.
AUDITORIA_PRECIO	precio_nuevo	NUMERIC(14,2)	No		Nuevo precio registrado.
AUDITORIA_PRECIO	fecha_hora	TIMESTAMP	No		Instante en el que se registró el cambio.
AUDITORIA_PRECIO	usuario	VARCHAR(100)	No		Usuario de BD que efectuó el cambio.

INFORME FASE 2

Entorno de prueba: PostgreSQL 16 en Docker – Base de Datos temporal: bodegas_atuel_paso7 (aunque también la usamos para el paso 8)

1. Objetivo y Alcance

Comparar dos sesiones PostgreSQL que compiten por existencias: Una lectura ordinaria sin bloqueo preventivo y otra mediante SELECT ... FOR UPDATE. Se presentan una simulación auxiliar que exhibe el riesgo de decisiones sobre lecturas antiguas, una prueba sobre la última botella en la tabla real stock_sala y una validación del procedimiento sp_procesar_transferencia.

El aislamiento utilizado fue READ COMMITTED. Las operaciones directas sobre stock_sala simulan el descuento de una venta; no se generaron comprobantes comerciales

2. Lectura sin bloqueo preventivo:

Se creó la tabla independiente demo_sin_bloqueo.stock con 10 botellas y 0 salidas autorizadas. Ambas sesiones abrieron transacciones y leyeron 10 mediante SELECT ordinario. Una lógica deliberadamente insegura basó las salidas en ese valor anterior y no volvió a verificar las existencias

```
PS C:\Users\Lasag\OneDrive\Documentos\AaaTINO\Universidad\3° Año\Base de Datos\Proyecto Integrador\trabaj
o_integrador_bd> docker exec -it pg-bodega-paso7 psql -U postgres -d bodegas_atuel_paso7
psql (16.15 (Debian 16.15-1.pgdg13+2))
Type "help" for help.

bodegas_atuel_paso7=# CREATE SCHEMA demo_sin_bloqueo;
CREATE SCHEMA
bodegas_atuel_paso7=# CREATE TABLE demo_sin_bloqueo.stock (
    id INTEGER PRIMARY KEY,
    cantidad INTEGER NOT NULL,
    salidas_autorizadas INTEGER NOT NULL
);
CREATE TABLE
bodegas_atuel_paso7=# INSERT INTO demo_sin_bloqueo.stock
VALUES (1, 10, 0);
INSERT 0 1
```

Creación de la tabla auxiliar y carga del stock inicial (10 botellas)

```
PS C:\Users\Lasag\OneDrive\Documentos\AaaTINO\Universidad\3° Año\Base de Datos\Proyecto Integrador\trabaj
o_integrador_bd> docker exec -it pg-bodega-paso7 psql -U postgres -d bodegas_atuel_paso7
psql (16.15 (Debian 16.15-1.pgdg13+2))
Type "help" for help.

bodegas_atuel_paso7=# BEGIN;
BEGIN
bodegas_atuel_paso7=*# SELECT cantidad
FROM demo_sin_bloqueo.stock
WHERE id = 1;
 cantidad
----------
       10
(1 row)
```

Lectura inicial de 10 botellas desde una sesión sin bloqueo preventivo

```
bodegas_atuel_paso7=*# UPDATE demo_sin_bloqueo.stock
SET cantidad = 3,
    salidas_autorizadas = salidas_autorizadas + 7
WHERE id = 1;
UPDATE 1
bodegas_atuel_paso7=*# COMMIT;
COMMIT
```

Actualización y confirmación de una de las transacciones

```
bodegas_atuel_paso7=# INSERT INTO demo_sin_bloqueo.stock
VALUES (1, 10, 0);
INSERT 0 1
bodegas_atuel_paso7=# SELECT
    cantidad AS stock_registrado,
    salidas_autorizadas,
    10 - salidas_autorizadas AS stock_real_calculado
FROM demo_sin_bloqueo.stock
WHERE id = 1;
 stock_registrado | salidas_autorizadas | stock_real_calculado
------------------+---------------------+----------------------
                3 |                  14 |                   -4
(1 row)
```

Resultado observado: 3 botellas registradas y 14 salidas autorizadas; salida lógica de -4

2.1. Ultima botella en stock_sala, sin FOR UPDATE

En una nueva base de pruebas se preparó el vino 1 en la sala 1 con cantidad = 1. A y B hicieron SELECT sin FOR UPDATE y ambas observaron 1. Luego A descontó la unidad con una operación condicionada a cantidad >= 1; B intentó el mismo UPDATE y esperó a que A confirmara. Al continuar, la condición se volvió a evaluar y B obtuvo UPDATE 0

3. Ultima botella sin FOR UPDATE

```
bodegas_atuel_ultima_botella=# BEGIN;

SHOW transaction_isolation;

SELECT cantidad
FROM stock_sala
WHERE id_sala = 1 AND cod_vino = 1;
BEGIN
 transaction_isolation
-----------------------
 read committed
(1 row)

 cantidad
----------
        1
(1 row)
```

Sesión A: Aislamiento READ COMMITED y lectura de 1 unidad

```
PS C:\Users\Lasag\OneDrive\Documentos\AaaTINO\Universidad\3° Año\Base de Datos\Proyecto Integrador\trabajo_integrador_bd> docker exec -it pg-bodeg
a-paso7 psql -U postgres -d bodegas_atuel_ultima_botella
psql (16.15 (Debian 16.15-1.pgdg13+2))
Type "help" for help.

bodegas_atuel_ultima_botella=# BEGIN;

SELECT cantidad
FROM stock_sala
WHERE id_sala = 1 AND cod_vino = 1;
BEGIN
 cantidad
----------
        1
(1 row)
```

Sesión B: También observa 1 unidad antes de la modificación

```
bodegas_atuel_ultima_botella=*# UPDATE stock_sala
SET cantidad = cantidad - 1
WHERE id_sala = 1
  AND cod_vino = 1
  AND cantidad >= 1
RETURNING cantidad;
 cantidad
----------
        0
(1 row)

UPDATE 1
```

Sesion A: UPDATE 1, stock descontando a 0

```
bodegas_atuel_ultima_botella=*# UPDATE stock_sala
SET cantidad = cantidad - 1
WHERE id_sala = 1
  AND cod_vino = 1
  AND cantidad >= 1
RETURNING cantidad;
 cantidad
----------
(0 rows)

UPDATE 0
```

Sesion B: UPDATE 0 tras confirmarse el descuento de A

En este experimento no hubo doble descuento. Aunque ambas sesiones leyeron 1, el UPDATE adquiere bloqueo de fila y la condición sobre las existencias impidió descontar una segunda unidad. Por eso debe distinguirse este resultado de la simulación auxiliar insegura de la sección 2

4. SELECT … FOR UPDATE

En la simulación auxiliar original se restablecieron 10 botellas. A ejecutó SELECT ... FOR UPDATE y mantuvo la transacción abierta; B intentó bloquear la misma fila y quedó esperando. Se observó la espera con pg_stat_activity y pg_blocking_pids(). Al hacer COMMIT A, B continuó y leyó 10, porque A solo había bloqueado la fila sin descontar unidades

```
bodegas_atuel_paso7=# BEGIN;
BEGIN
bodegas_atuel_paso7=*# SELECT cantidad
FROM demo_sin_bloqueo.stock
WHERE id = 1
FOR UPDATE;
```

La sesión A lee con FOR UPDATE y retiene el bloqueo

```
bodegas_atuel_paso7=# BEGIN;
BEGIN
bodegas_atuel_paso7=*# SELECT cantidad
FROM demo_sin_bloqueo.stock
WHERE id = 1
FOR UPDATE;
```

La sesión B intenta adquirir el mismo bloqueo

5. Ultima botella con FOR UPDATE

Se restituyó cantidad = 1 en stock_sala. A inició una transacción y realizó SELECT ... FOR UPDATE, obteniendo 1 y reteniendo el bloqueo. B ejecutó la misma consulta bloqueante y quedó esperando antes de poder tomar una decisión sobre la venta

```
bodegas_atuel_ultima_botella=# UPDATE stock_sala
SET cantidad = 1
WHERE id_sala = 1
  AND cod_vino = 1;

SELECT id_sala, cod_vino, cantidad
FROM stock_sala
WHERE id_sala = 1
  AND cod_vino = 1;
UPDATE 1
 id_sala | cod_vino | cantidad
---------+----------+----------
       1 |        1 |        1
(1 row)

bodegas_atuel_ultima_botella=# BEGIN;

SHOW transaction_isolation;

SELECT cantidad
```

Se restablece una unidad para repetir el escenario

```
SELECT cantidad
FROM stock_sala
WHERE id_sala = 1
  AND cod_vino = 1
FOR UPDATE;
BEGIN
 transaction_isolation
-----------------------
 read committed
(1 row)

 cantidad
----------
        1
(1 row)
```

Sesion A READ COMMITED y SELECT … FOR UPDATE devuelve 1

Durante la espera de B, A realizó un UPDATE condicionado, descontó la única unidad y confirmó. Solo entonces B obtuvo el bloqueo y pudo leer el valor actualizado.

```
bodegas_atuel_ultima_botella=*# COMMIT;

SELECT id_sala, cod_vino, cantidad
FROM stock_sala
WHERE id_sala = 1 AND cod_vino = 1;
COMMIT
 id_sala | cod_vino | cantidad
---------+----------+----------
       1 |        1 |        0
(1 row)

bodegas_atuel_ultima_botella=# BEGIN;

SELECT cantidad
FROM stock_sala
WHERE id_sala = 1
  AND cod_vino = 1
FOR UPDATE;
BEGIN
 cantidad
----------
        0
(1 row)
```

Sesion B luego de la confirmación de A, FOR UPDATE devuelve 0

```
SELECT CASE
    WHEN cantidad >= 1 THEN 'VENTA PERMITIDA'
    ELSE 'VENTA RECHAZADA: STOCK INSUFICIENTE'
END AS resultado
FROM stock_sala
WHERE id_sala = 1
  AND cod_vino = 1;

COMMIT;

-- Verificación final
SELECT id_sala, cod_vino, cantidad
FROM stock_sala
WHERE id_sala = 1
  AND cod_vino = 1;
             resultado
-------------------------------------
 VENTA RECHAZADA: STOCK INSUFICIENTE
(1 row)

COMMIT
 id_sala | cod_vino | cantidad
---------+----------+----------
       1 |        1 |        0
(1 row)
```

Decision de B venta rechazada por stock insuficiente, saldo final 0

La segunda sesión no intentó descontar la botella después de observar 0: la decisión de no vender se tomó con el valor actualizado. El experimento evidencia lectura bloqueante y coordinación entre transacciones

6. Aplicación en el procedimiento del proyecto

En una prueba independiente con el procedimiento real sp_procesar_transferencia, las transferencias T1=2 y T2=3 intentaron mover 7 botellas cada una desde un depósito con 10. Se observó que B esperaba a A. Cuando A confirmó, B fue rechazada por RN-11 (stock insuficiente: disponible 3, solicitado 7). La verificación final confirmó T1 procesado, T2 terminado, depósito 3 y sala 7. Esta evidencia complementa la simulación auxiliar.

```
PS C:\Users\Lasag\OneDrive\Documentos\AaaTINO\Universidad\3° Año\Base de Datos\Proyecto Integrador\trabaj
o_integrador_bd> docker exec -it pg-bodega-paso7 psql -U postgres -d bodegas_atuel_paso7
psql (16.15 (Debian 16.15-1.pgdg13+2))
Type "help" for help.

bodegas_atuel_paso7=# SELECT
    pid,
    state,
    wait_event_type,
    wait_event,
    pg_blocking_pids(pid) AS bloqueado_por
FROM pg_stat_activity
WHERE datname = 'bodegas_atuel_paso7'
ORDER BY pid;
 pid |        state        | wait_event_type |  wait_event   | bloqueado_por
-----+---------------------+-----------------+---------------+---------------
 242 | idle                | Client          | ClientRead    | {}
 251 | idle                | Client          | ClientRead    | {}
 259 | idle                | Client          | ClientRead    | {}
 334 | idle                | Client          | ClientRead    | {}
 347 | idle in transaction | Client          | ClientRead    | {}
 355 | active              | Lock            | transactionid | {347}
 382 | active              |                 |               | {}
(7 rows)
```

Monitoreo de una sesión en espera mediante pg_stat_activity y pg_blocking_pids()

6.1. Nivel de aislamiento utilizado

Se utilizó READ COMMITTED, el aislamiento predeterminado de PostgreSQL. Las consultas ordinarias ven datos confirmados al comienzo de cada sentencia, pero varias sesiones pueden tomar decisiones a partir de lecturas que luego quedan desactualizadas. SELECT ... FOR UPDATE espera ante un bloqueo incompatible y protege la fila hasta finalizar la transacción. El uso de una actualización condicionada y de controles de negocio sigue siendo necesario.

```
PS C:\Users\Lasag\OneDrive\Documentos\AaaTINO\Universidad\3° Año\Base de Datos\Proyecto Integrador\trabaj
o_integrador_bd> docker exec -it pg-bodega-paso7 psql -U postgres -d bodegas_atuel_paso7
psql (16.15 (Debian 16.15-1.pgdg13+2))
Type "help" for help.

bodegas_atuel_paso7=# SHOW transaction_isolation;
 transaction_isolation
-----------------------
 read committed
(1 row)

bodegas_atuel_paso7#
```

Confirmacion de aislamiento mediante SHOW transacción_isolation: read commited

7. Comparación y conclusión

Situación	Lo observado	Implicación
Simulación auxiliar sin bloqueo	Dos lecturas iguales; lógica insegura registra 14 salidas sobre 10	Riesgo de decidir con lectura antigua
Ultima botella sin FOR UPDATE	Ambas leen 1; A hace UPDATE 1; B espera y obtiene UPDATE 0	El UPDATE bloquea y reevalua la condición
Ultimas botellas con FOR UPDATE	A lee y bloquea 1; B espera; luego observa 0	La decisión se toma con stock actualizado
Procedimiento real de transferencia	Dos solicitudes de 7 sobre deposito de 10; solo una se proceso	La lógica transaccional rechaza faltante de stock

Las pruebas muestran que FOR UPDATE anticipa la coordinación desde la lectura; sin él, dos sesiones pueden leer el mismo stock, aunque los UPDATE siguen adquiriendo bloqueos. La simulación auxiliar muestra el problema de programar ventas a partir de lecturas antiguas sin una verificación segura. Los procedimientos del proyecto y las restricciones de stock, por su parte, evitan que las transferencias concurrentes produzcan saldos negativos o cambios parciales.

Alcance de la evidencia: el caso de última botella se reprodujo directamente sobre stock_sala, sin comprobantes. La prueba de transferencia corresponde al procedimiento real sp_procesar_transferencia. Todos los ensayos se realizaron en bases de pruebas separadas, sin alterar las migraciones del proyecto

TABLA DE TRAZABILIDAD DE REGLAS DE NEGOCIO – FASE 2

La siguiente tabla relaciona las 15 reglas de negocio definidas en el enunciado con los mecanismos implementados en la base de datos y los casos de prueba utilizados para verificar su cumplimiento. Los casos se encuentran en tests/db_tests.sql y fueron ejecutados satisfactoriamente sobre PostgreSQL 16.

RN	Regla de negocio	Mecanismo implementado	Caso de prueba
RN-01	Precio único por vino y mayor que cero.	vino.precio; CHECK chk_vino_precio (002_constraints.sql).	B: precio cero → 23514; vino duplicado → 23505.
RN-02	Cada sala pertenece a una localidad; encargado regional por localidad.	FK fk_sala_venta_localidad; esquema normalizado (001_esquema.sql).	B: sala con localidad inexistente → 23503
RN-03	Cada empleado trabaja en una sola sala.	empleado.id_sala NOT NULL; FK fk_empleado_sala_venta.	B: empleado sin sala → 23502.
RN-04	Una cepa por parcela y una cosecha máxima por campaña.	parcela.id_cepa; FK y UNIQUE uq_lote_uva_parcela_campania.	B: dos cosechas en misma parcela/campaña → 23505.
RN-05	Los porcentajes de composición de un vino suman 100 %.	Constraint triggers diferidos trg_vino_composicion_obligatoria y trg_vino_cepa_sum100 (005).	E: 99,99 % y vino sin cepas rechazados; mezcla 60/40 aceptada.
RN-06	Descuento según tipo de cliente.	fn_descuento_cliente (003); CHECK sobre tipo_cliente (002).	C: socio 10 %, distribuidor 20 %, exportador 30 %.
RN-07	Permiso de exportación vence después de su emisión.	CHECK chk_permiso_fechas (002).	B: vencimiento igual a emisión → 23514.
RN-08	Importe del comprobante calculado según renglones y descuentos.	fn_total_comprobante (003); triggers de comprobante y renglones (005).	C/E: comprobante vacío = 0; venta: subtotal 200, descuento 20, total 180.
RN-09	No vender más unidades que las disponibles en sala.	trg_renglon_comprobante (005); FOR UPDATE; CHECK stock_sala ≥ 0.	E: venta de 2 unidades aceptada; 9999 unidades rechazadas → P0001.
RN-10	Ingresos: estados válidos, proceso único e impacto en depósito.	sp_terminar_ingreso / sp_procesar_ingreso (004); trigger de estados (005).	D: termina/procesa ingreso y suma 10; reprocesamiento rechazado → P0001.
RN-11	Transferencias: estados válidos y fecha inmutable.	sp_terminar_transferencia / sp_procesar_transferencia (004); triggers de estados y fecha (005).	D: transfiere 5, fecha actual; rechaza cambio de fecha, retroceso y reprocesamiento.
RN-12	No editar renglones cuando el documento salió de confección.	trg_renglon_ingreso_estado y trg_renglon_transferencia_estado (005).	D: modificación de renglón de ingreso procesado rechazada → P0001.
RN-13	Auditar cambios efectivos en precio.	trg_auditoria_precio_vino; tabla auditoria_precio (005/001).	E: cambio de precio registrado; UPDATE sin cambio no añade auditoría.
RN-14	No borrar vinos, bodegas ni transportistas con movimientos.	FK ON DELETE RESTRICT (002_constraints.sql).	B/D: eliminar vino, bodega o transportista referenciados → 23503.
RN-15	Cantidades, superficies, kilos y precios positivos; graduación entre 5 y 20.	CHECK de valores/rangos (002_constraints.sql).	B/D: superficie negativa, graduación 25, cantidad negativa → 23514.

INFORME FASE 3

Este documento presenta la Etapa 3 conforme a la consigna oficial (págs. 13–14). Los resultados provienen de ejecuciones y controles en PostgreSQL 18.1. La reproducibilidad ya fue verificada mediante una segunda carga small y comparación de las 27 tablas. El procedimiento de carga, sus requisitos y las instrucciones de ejecución se documentan en db/seed/README.md.

1. Objetivo y Alcance

Se desarrolló una carga automatizada de datos para PostgreSQL, con información de catálogos, producción vitivinícola, movimientos logísticos, ventas y auditoría. El objetivo de volumen fijado por la cátedra exige poblar todas las tablas, superar las 10.000 filas en al menos tres de ellas y las 100.000 en al menos una.

Se utilizaron los perfiles small y full. La carga full fue ejecutada sobre una base nueva denominada bodegas_atuel_fase3_full_final, con las siete migraciones aplicadas y con las claves foráneas, restricciones y triggers activos.

2. Catálogos, scripts y orden de carga

Los catálogos de cepas, maridajes, localidades y departamentos de Mendoza y tipos de vino se incorporan desde archivos CSV. Sus fuentes y criterios de selección están documentados en db/seed/README.md; los datos de referencia no se generan mediante Faker. Los datos ficticios de bodegas, fincas, clientes y operaciones se generan con parámetros y generadores pseudoaleatorios.

La implementación se organiza en db/seed/: config.py contiene perfiles y conexiones; run_seed.py coordina la ejecución; control_integridad.sql agrupa consultas de validación; prueba_rendimiento.py compara dos estrategias de inserción; los cuatro módulos de carga están en db/seed/modules/.

2.1 Esquema de dependencias

El orden de los módulos respeta las dependencias de claves foráneas. Las referencias reales se comparten en un diccionario refs para evitar suponer identificadores generados por las secuencias de PostgreSQL.

CATALOGOS → PRODUCCION → LOGISTICA → VENTAS → RN-13

localidad → sala_venta → transferencia → renglon_transferencia
localidad → cliente → comprobante → renglon_comprobante
finca → parcela → lote_uva → analisis_calidad
bodega → vino → vino_cepa
vino → auditoria_precio

En términos de tablas, los catálogos y maestros preceden a fincas, parcelas, lotes y vinos; luego se procesan ingresos y transferencias para generar existencias, por último, se cargan clientes, comprobantes y renglones de venta. Tras ventas se efectúa una actualización de precio para generar la auditoría RN-13.

3. Coherencia de datos y estrategia frente a triggers

La generación utiliza semillas independientes por módulo, de forma que los datos aleatorios de uno no alteran la secuencia de los siguientes. Las cosechas, análisis, lotes y vinos conservan relaciones coherentes; la composición vino-cepa se genera con porcentajes que suman 100 %. Los clientes se asocian a localidades y tipos válidos; los permisos corresponden a clientes exportadores.

Se mantuvieron activos los triggers, las claves foráneas y las restricciones. Los ingresos y las transferencias se procesaron mediante los procedimientos de la base; no se modificaron directamente las existencias para evitar duplicar movimientos. Los descuentos, subtotales y totales son gestionados por los mecanismos de la base, y los renglones vendidos descuentan stock mediante triggers.

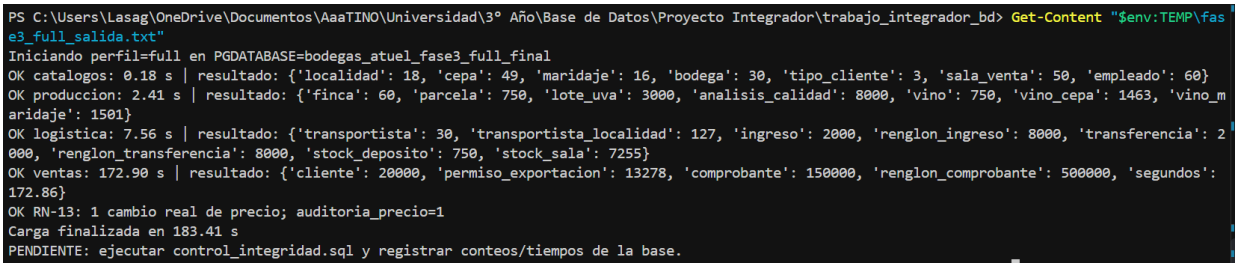
Esta decisión permite comprobar las mismas reglas que usaría el sistema en funcionamiento. Su principal costo es el tiempo de ejecución —sobre todo en ventas, por la cantidad de renglones y actualizaciones relacionadas— y el riesgo de rechazos si faltan existencias. Deshabilitar triggers habría simplificado la carga histórica, pero exigiría reconstruir y validar manualmente sus efectos.

La RN-13 se validó mediante una actualización real sobre un vino: UPDATE vino SET precio = precio + 1.00. El trigger creó una fila en auditoria_precio. La actualización se hace después de registrar las ventas, por lo que no reescribe los precios unitarios ya almacenados en comprobantes.

Respecto del tiempo, las transferencias y ventas se registran en la fecha operativa de PostgreSQL (CURRENT_DATE) debido a las restricciones de la RN-11. Por eso el historial de producción usa fechas previas y la reproducibilidad de fechas operativas se compara dentro del mismo día.

4. Carga full: Volúmenes y Tiempos reales

La siguiente evidencia corresponde a la ejecución de run_seed.py --run en la base final, con PROFILE=full. El programa confirmó la ejecución de catálogos, producción, logística, ventas y la auditoría RN-13; la carga completa tardó 183,41 segundos. El módulo de ventas concentró el mayor tiempo debido a sus 150.000 comprobantes y 500.000 renglones.



Ejecución de la carga full por módulos y confirmación de RN-13

En lugar de duplicar los conteos en una tabla de Word, a continuación, se muestra la salida del control SQL que informa cada tabla y los tres resúmenes de volumen. Se obtuvo un resultado OK en las 27 tablas; cuatro superaron 10.000 filas y dos superaron 100.000.

control	cantidad	estado
analisis_calidad	8000	OK
auditoria_precio	1	OK
bodega	30	OK
cepa	49	OK
cliente	20000	OK
comprobante	150000	OK
empleado	60	OK
finca	60	OK
ingreso	2000	OK
localidad	18	OK
lote_uva	3000	OK
maridaje	16	OK
parcela	750	OK
permiso_exportacion	13278	OK
renglon_comprobante	500000	OK
renglon_ingreso	8000	OK
renglon_transferencia	8000	OK
sala_venta	50	OK
stock_deposito	750	OK
stock_sala	7255	OK
tipo_cliente	3	OK
transferencia	2000	OK
transportista	30	OK
transportista_localidad	127	OK
vino	750	OK
vino_cepa	1463	OK
vino_maridaje	1501	OK
RESUMEN: tablas con al menos una fila	27	OK
RESUMEN: tablas con mas de 10000 filas	4	OK
RESUMEN: tablas con mas de 100000 filas	2	OK

Conteo de las 27 tablas y cumplimiento de los mínimos de volumen

5. Consultas de control e integridad

Se ejecutó db/seed/control_integridad.sql sobre la base full y posteriormente sobre la base restaurada. El archivo comprueba los conteos, las referencias huérfanas, el stock negativo y las diferencias entre importes guardados y recalculados. Las consultas completas quedan disponibles en el repositorio, en vez de copiar su SQL extenso dentro del informe.

5.1. Integridad referencial

La consulta verificó 32 relaciones de claves foráneas y devolvió cero registros huérfanos en todos los casos. La captura siguiente muestra la tabla de relaciones; la salida completa terminó con “(32 rows)”.

relacion	huerfanos	estado
fk_analisis_calidad_lote_uva	0	OK
fk_auditoria_precio_vino	0	OK
fk_cliente_localidad	0	OK
fk_cliente_tipo_cliente	0	OK
fk_comprobante_cliente	0	OK
fk_comprobante_sala_venta	0	OK
fk_empleado_sala_venta	0	OK
fk_ingreso_bodega	0	OK
fk_ingreso_transportista	0	OK
fk_lote_uva_bodega	0	OK
fk_lote_uva_parcela	0	OK
fk_parcela_cepa	0	OK
fk_parcela_finca	0	OK
fk_permiso_exportacion_cliente	0	OK
fk_renglon_comprobante_comprobante	0	OK
fk_renglon_comprobante_vino	0	OK
fk_renglon_ingreso_ingreso	0	OK
fk_renglon_ingreso_vino	0	OK
fk_renglon_transferencia_transferencia	0	OK
fk_renglon_transferencia_vino	0	OK
fk_sala_venta_localidad	0	OK
fk_stock_deposito_vino	0	OK
fk_stock_sala_sala_venta	0	OK
fk_stock_sala_vino	0	OK
fk_transferencia_sala_venta	0	OK

```
fk_vino_cepa_cepa                    |         0 | OK
fk_vino_cepa_vino                    |         0 | OK
fk_vino_maridaje_maridaje            |         0 | OK
fk_vino_maridaje_vino                |         0 | OK
(32 rows)
```

Control de integridad referencial de las claves foráneas

5.2. Existencias e importes comerciales

Los controles de stock_deposito y stock_sala devolvieron cero filas con cantidades negativas. La consulta que compara subtotal, descuento y total de los comprobantes con los importes esperados no devolvió registros (0 rows).

```
     tabla      | filas_negativas | estado
----------------+-----------------+--------
 stock_deposito |               0 | OK
 stock_sala     |               0 | OK
(2 rows)

 nro_comprobante | subtotal_guardado | subtotal_esperado | descuento_guardado | descuento_esperado | total_guardado | total_esperado | total_renglon
es_guardados
-----------------+-------------------+-------------------+--------------------+--------------------+----------------+----------------+-------------
-------------
(0 rows)
```

Controles de stock negativo y comprobantes con importes inconsistentes

5.3. Conservación de stock por sala y vino

Además, se realizó una conciliación independiente por cada combinación de sala y vino, considerando las transferencias procesadas, los renglones vendidos y el stock final. Se verificó la igualdad 89.537.974 - 999.954 = 88.538.020, con cero diferencias en el conjunto de combinaciones

```
 botellas_transferidas | botellas_vendidas | stock_actual | diferencias_stock
-----------------------+-------------------+--------------+-------------------
              89537974 |            999954 |     88538020 |                 0
(1 row)
```

Conciliación de transferencias, ventas y existencias finales

En la validación específica de ventas también se comprobaron documentos de cliente no duplicados, permisos válidos de exportación, descuentos correctos, ausencia de vinos repetidos por comprobante y ventas respaldadas por transferencias previas compatibles; no se informaron inconsistencias en estas consultas.

6. Comparación de rendimientos de inserción

La cátedra solicita comparar inserciones individuales con un método de carga masiva. Para la prueba principal se usó una tabla temporal basada en cliente (sin copiar FK ni triggers), con los mismos 10.000 registros en cada método. Se midieron tres repeticiones por INSERT fila a fila y COPY FROM STDIN; el cronómetro incluyó inserción y COMMIT, pero no preparación, limpieza ni COUNT.

```
Base: bodegas_atuel_rn13_small | PostgreSQL: 18.1
Muestra identica: 10000 filas | repeticiones: 3
Tabla temporal basada en cliente; sin FK ni triggers
Cronometro: insercion + COMMIT; limpieza y COUNT excluidos
Repeticion 1: fila_por_fila=3.399 s; COPY=0.103 s
Repeticion 2: fila_por_fila=3.510 s; COPY=0.134 s
Repeticion 3: fila_por_fila=3.501 s; COPY=0.148 s

RESULTADO (medianas)
Fila por fila: 3.501 s (10000 INSERT)
COPY:          0.134 s (1 operacion COPY)
Aceleracion COPY: 26.14x
Ahorro de tiempo: 96.2%
Verificacion: misma cantidad de filas en cada repeticion
Seguridad: no se modificaron tablas permanentes
Evidencia: C:\Users\Lasag\AppData\Local\Temp\prueba_rendimiento.csv
```

Mediciones de tres repeticiones: INSERT individuales frente a COPY

La mediana de INSERT individual fue 3,501 s, frente a 0,134 s con COPY. La carga mediante COPY resultó 26,14 veces más rápida, equivalente a una reducción del 96,2 % del tiempo observado. También se realizó una prueba separada con INSERT multi-VALUES en lotes de 1.000 registros: su mediana fue 0,448 s frente a 3,461 s para inserciones individuales. Los factores de ambas pruebas no deben confundirse con la duración de la carga full.

La diferencia observada se explica principalmente por la menor cantidad de operaciones y viajes de consulta al servidor. El ensayo aísla el mecanismo de inserción; los triggers y las restricciones utilizadas durante la carga real implican un costo adicional que no está incluido en esos tiempos.

7. Reproducibilidad con semilla fija

Se compararon dos ejecuciones independientes con PROFILE=small, SEED=20261027 y BATCH_SIZE=1000, realizadas en bodegas_atuel_rn13_small y bodegas_atuel_repro_small. La segunda terminó correctamente (código de salida 0).

Para ir más allá de los conteos, se calcularon huellas SHA-256 a partir de representaciones JSON de las filas ordenadas de cada tabla. Las 27 tablas tuvieron huellas coincidentes y la comparación final indicó cero tablas diferentes. Se excluyó únicamente auditoria_precio.fecha_hora, porque refleja el momento real de ejecución; las fechas dependientes de CURRENT_DATE coincidieron al realizar las pruebas el mismo día.

```
COMPARACION DE REPRODUCIBILIDAD
Semilla: 20261027 | Perfil: small
Auditoria: se excluye fecha_hora
------------------------------------------------------------
analisis_calidad                 50 /    50 IGUAL
auditoria_precio                  1 /     1 IGUAL
bodega                            5 /     5 IGUAL
cepa                             12 /    12 IGUAL
cliente                         100 /   100 IGUAL
comprobante                     100 /   100 IGUAL
empleado                          5 /     5 IGUAL
finca                             5 /     5 IGUAL
ingreso                          10 /    10 IGUAL
localidad                        10 /    10 IGUAL
lote_uva                         50 /    50 IGUAL
maridaje                         10 /    10 IGUAL
parcela                          25 /    25 IGUAL
permiso_exportacion              57 /    57 IGUAL
renglon_comprobante             250 /   250 IGUAL
renglon_ingreso                  20 /    20 IGUAL
renglon_transferencia            20 /    20 IGUAL
sala_venta                        5 /     5 IGUAL
stock_deposito                   15 /    15 IGUAL
stock_sala                       19 /    19 IGUAL
tipo_cliente                      3 /     3 IGUAL
transferencia                    10 /    10 IGUAL
transportista                     5 /     5 IGUAL
transportista_localidad          19 /    19 IGUAL
vino                             25 /    25 IGUAL
vino_cepa                        53 /    53 IGUAL
vino_maridaje                    47 /    47 IGUAL
------------------------------------------------------------
Tablas comparadas: 27
Tablas con diferencias: 0
RESULTADO: REPRODUCIBILIDAD COMPROBADA
```

Comparación reproducible de las tablas con la misma semilla

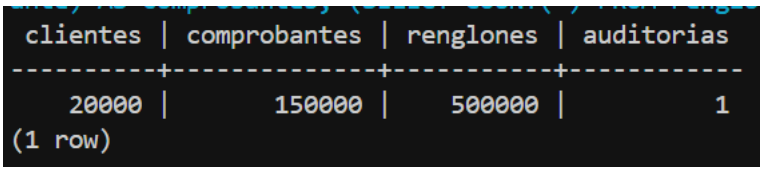
El cierre de la misma prueba indicó: «Tablas comparadas: 27», «Tablas con diferencias: 0» y «RESULTADO: REPRODUCIBILIDAD COMPROBADA». Este resultado avala la reproducibilidad de los datos deterministas de small bajo las condiciones documentadas; no implica igualdad byte a byte de los campos tomados del reloj en ejecuciones de días distintos.

8. Respaldo y recuperación

Se generó un volcado completo con pg_dump y se comprimió como dump_completo.sql.gz. El archivo tiene 10.502.108 bytes comprimidos y 31.002.722 bytes al descomprimir; la lectura completa del GZIP terminó con código 0. Se calculó la huella SHA-256 para identificar la copia utilizada:

D1A066940B9429866FB490D7CE5675159218BDB7F7376CB9582C490696DD0B78

El volcado se restauró en una base descartable nueva denominada bodegas_atuel_restauracion. Allí se comprobó la recuperación de 20.000 clientes, 150.000 comprobantes, 500.000 renglones y una fila de auditoría. La ejecución posterior de control_integridad.sql volvió a mostrar las 27 tablas pobladas, sin filas huérfanas, sin stock negativo ni comprobantes inconsistentes.



```
 clientes | comprobantes | renglones | auditorias
----------+--------------+-----------+------------
    20000 |       150000 |    500000 |          1
(1 row)
```

Conteos recuperados desde la base restaurada

9. Conclusión técnica

La Fase 3 alcanzó los criterios de volumen y consistencia verificados: 27 tablas pobladas, cuatro tablas con más de 10.000 registros, dos con más de 100.000, 500.000 renglones de venta, 32 controles de FK sin huérfanos, cero existencias negativas y cero inconsistencias de importes. La auditoría RN-13 contiene un cambio efectivo de precio, la conciliación de stock cierra sin diferencias y el dump comprimido fue restaurado correctamente.

La reproducibilidad se comprobó entre dos ejecuciones small con la misma semilla y 27 tablas coincidentes, dejando documentada la excepción del timestamp de auditoría y la dependencia de CURRENT_DATE.

10. Fuentes y ubicación de evidencias

Consigna: TPI Bodegas del Atuel, sección «Etapa 3. Carga masiva de datos», págs. 13–14. Código: repositorio github.com/videlalauti/trabajo_integrador_bd, rama main. Archivos relevantes: db/seed/README.md, config.py, run_seed.py, control_integridad.sql, modules/*.py y prueba_rendimiento.py.

Evidencias del equipo: fase3_full_salida.txt, fase3_full_integridad.txt, fase3_full_stock.txt, prueba_rendimiento.csv, prueba_rendimiento_salida.txt, reproducibilidad_fase3.txt, repro_small_salida.txt, dump_completo.sql.gz y salida de restauración. Los archivos de evidencia y el dump se conservan fuera del repositorio hasta definir el paquete final de entrega.

INFORME FASE 4

En esta fase probamos cinco índices para comprobar si mejoraban el rendimiento de algunas consultas. Comparamos los planes de ejecución antes y después, analizamos los índices que no resultaron útiles y medimos cuánto costó mantener índices adicionales al insertar registros. Por último, verificamos que los cinco índices funcionaran juntos.

1. Objetivo y Alcance

El objetivo fue elegir consultas del proyecto, medir cómo funcionaban y decidir qué índices convenía agregar. Nos guiamos por la Etapa 4 de la consigna (páginas 14 y 15) y por las consultas de la sección 8.

Utilizamos la base cargada en la Fase 3, con 27 tablas que contienen datos. Para estas pruebas resultaron importantes comprobante (150.000 filas), renglon_comprobante (500.000), cliente (20.000), transferencia (2.000) y renglon_transferencia (8.000). Cada integrante trabajó sobre una copia de prueba, sin modificar la base original ni el respaldo.

2. Entorno y metodología de evaluación

Al comenzar había 33 índices generados por las claves primarias y las restricciones UNIQUE. Las pruebas se realizaron con PostgreSQL 18, en distintas versiones según el equipo (18.1, 18.3 y 18.6). Por eso comparamos los tiempos antes y después dentro de cada prueba, sin comparar directamente la velocidad de las computadoras de los integrantes.

Primero actualizamos las estadísticas con ANALYZE; en las pruebas de Antonio también se utilizó VACUUM ANALYZE. Después ejecutamos EXPLAIN (ANALYZE, BUFFERS) antes y después de crear cada índice. Hicimos tres mediciones por escenario y usamos la mediana del tiempo de ejecución. No obligamos a PostgreSQL a usar índices: comprobamos en cada plan si realmente los elegía.

Calculamos la mejora como (tiempo antes − tiempo después) / tiempo antes × 100. Para considerar que la mejora se debía al índice, también revisamos que el plan lo utilizara, y no solo que el tiempo fuera menor.

Un detalle importante de los datos es que todos los comprobantes comerciales tienen fecha 30/10/2026. Por eso algunos filtros de fechas, como “último trimestre”, no ayudan a reducir la cantidad de registros que se deben consultar.

3. Diseño e integración de índices

Índice definitivo	Tabla y definición principal	Justificación
idx_comprobante_sala_fecha	comprobante (id_sala, fecha)	Compuesto: igualdad por sala y rango temporal en S3.
idx_rc_cod_vino	renglon_comprobante (cod_vino)	FK: acceso selectivo por vino; evaluado con vino 442.
idx_comprobante_id_cliente_inc	comprobante (id_cliente) INCLUDE (nro_comprobante)	Especial / FK: acceso cubierto para S12-A.
idx_rt_cod_vino_nro	renglon_transferencia (cod_vino, nro_solicitud)	Compuesto: buscar vino y relacionar transferencia en S4.
idx_transferencia_pendiente_sala	transferencia (id_sala) WHERE estado <> ‘procesado’	Parcial: filas aún no procesadas, S5; ausencia de casos positivos en dump.

En el índice de S3 colocamos primero id_sala, porque la consulta busca una sala específica, y después fecha, porque filtra por un período. En S4 usamos (cod_vino, nro_solicitud): no es igual a la clave primaria existente (nro_solicitud, cod_vino), ya que la consulta comienza buscando por vino. Los índices que no aportaron mejoras no se incluyeron en la migración definitiva.

Los cinco índices quedaron definidos en db/migrations/008_indices.sql. Al aplicarlos en la base de integración, la consulta al catálogo pg_indexes confirmó 38 índices en total: los 33 anteriores y los cinco nuevos.

4. Comparación de consultas oficiales

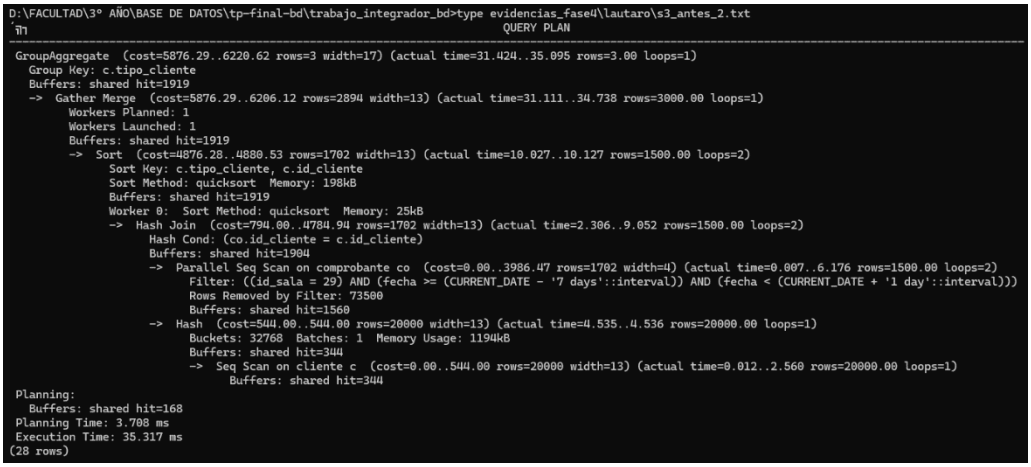
Consulta	Antes (ms)	Después (ms)	Mejora	Recorrido	Filas revisadas / descartadas
S3 – Sala 29	35,317	12,623	64,26%	Parallel Seq Scan → Bitmap Index Scan	150.000 revisadas; 147.000 descartadas → 3.000 encontradas
S4 – Vino 681	1,041	0,719	30,93%	Seq Scan → Bitmap Index Scan	8.000 revisadas; 7.980 descartadas → 20 encontradas
S12-A – Bodega DEMO Atuel 001	104.823,719	365,214	99,65%	Seq Scan reiterado → Index Only Scan	20.370 recorridos; ~148.592 descartadas por recorrido → búsquedas por cliente

La tabla muestra las medianas de las pruebas individuales. También indicamos cómo cambió el acceso a las tablas y cuántas filas se revisaron o descartaron, según lo que informa cada plan. Más adelante probamos los cinco índices juntos para verificar que siguieran utilizándose.

4.1. S3 – Clientes por tipo en una sala durante la última semana

Para la sala 29, sin el índice PostgreSQL recorría los 150.000 comprobantes y descartaba 147.000 para encontrar los 3.000 que cumplían el filtro. Después de crear idx_comprobante_sala_fecha, pasó a buscar primero en el índice y luego a consultar los bloques de tabla necesarios. La unión con cliente siguió siendo la misma.

El tiempo bajó de 35,317 ms a 12,623 ms: una mejora del 64,26 %. En esta base, la diferencia se explica principalmente por la selección de la sala, porque todos los comprobantes tienen la misma fecha.



S3 antes: recorrido secuencial paralelo sobre comprobante.

```
D:\FACULTAD\3° AÑO\BASE DE DATOS\tp-final-bd\trabajo_integrador_bd>type evidencias_fase4\lautaro\s3_despues_1.txt
                                                                 QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------------------------------------
 GroupAggregate  (cost=2719.46..2741.19 rows=3 width=17) (actual time=12.173..12.398 rows=3.00 loops=1)
   Group Key: c.tipo_cliente
   Buffers: shared hit=1897 read=14
   ->  Sort  (cost=2719.46..2726.69 rows=2894 width=13) (actual time=12.044..12.138 rows=3000.00 loops=1)
         Sort Key: c.tipo_cliente, c.id_cliente
         Sort Method: quicksort  Memory: 198kB
         Buffers: shared hit=1897 read=14
         ->  Hash Join  (cost=879.33..2553.07 rows=2894 width=13) (actual time=5.417..10.340 rows=3000.00 loops=1)
               Hash Cond: (co.id_cliente = c.id_cliente)
               Buffers: shared hit=1891 read=14
               ->  Bitmap Heap Scan on comprobante co  (cost=85.33..1751.47 rows=2894 width=4) (actual time=0.610..4.856 rows=3000.00 loops=1)
                     Recheck Cond: ((id_sala = 29) AND (fecha >= (CURRENT_DATE - '7 days'::interval)) AND (fecha < (CURRENT_DATE + '1 day'::interval)))
                     Heap Blocks: exact=1547
                     Buffers: shared hit=1547 read=14
                     ->  Bitmap Index Scan on idx_comprobante_sala_fecha  (cost=0.00..84.61 rows=2894 width=0) (actual time=0.466..0.467 rows=3000.00 loops=1)
                           Index Cond: ((id_sala = 29) AND (fecha >= (CURRENT_DATE - '7 days'::interval)) AND (fecha < (CURRENT_DATE + '1 day'::interval)))
                           Index Searches: 1
                           Buffers: shared read=14
               ->  Hash  (cost=544.00..544.00 rows=20000 width=13) (actual time=4.679..4.679 rows=20000.00 loops=1)
                     Buckets: 32768  Batches: 1  Memory Usage: 1194kB
                     Buffers: shared hit=344
                     ->  Seq Scan on cliente c  (cost=0.00..544.00 rows=20000 width=13) (actual time=0.023..2.615 rows=20000.00 loops=1)
                           Buffers: shared hit=344
 Planning:
   Buffers: shared hit=200 read=1
 Planning Time: 4.802 ms
 Execution Time: 12.623 ms
(27 rows)
```

S3 después: Bitmap Index Scan usando el índice compuesto.

4.2. S4 – Salas que solicitaron mas unidades de un vino

Para el vino 681 encontramos 20 renglones de transferencia, correspondientes a 20 solicitudes de 18 salas. Sin índice, PostgreSQL revisaba los 8.000 renglones y descartaba 7.980. Con idx_rt_cod_vino_nro pasó a localizar los 20 renglones mediante el índice.

El tiempo bajó de 1,041 ms a 0,719 ms, una mejora del 30,93 %. Aunque la diferencia es de solo 0,322 ms, el plan muestra que se evitaron miles de comprobaciones innecesarias.

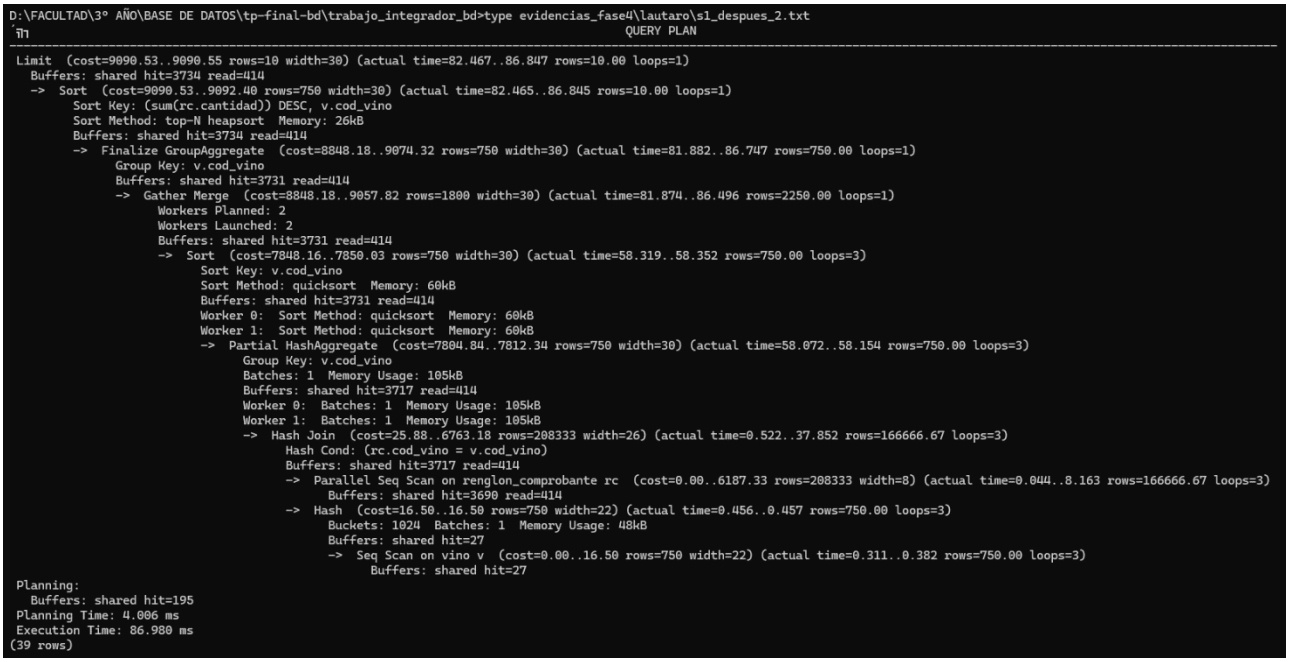
```
PS C:\Users\joseg\OneDrive\Documentos\3\Base de Datos\TRABAJO FINAL\trabajo_integrador_bd> Get-Content .\fase4\evidencias\jose\s4_antes_1.txt
                                                                 QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------------------------------------
 Limit  (cost=211.63..211.64 rows=5 width=12) (actual time=0.943..0.946 rows=5.00 loops=1)
   Buffers: shared hit=61
   ->  Sort  (cost=211.63..211.68 rows=20 width=12) (actual time=0.942..0.944 rows=5.00 loops=1)
         Sort Key: (sum(rt.cantidad)) DESC, t.id_sala
         Sort Method: top-N heapsort  Memory: 25kB
         Buffers: shared hit=61
         ->  GroupAggregate  (cost=210.94..211.29 rows=20 width=12) (actual time=0.909..0.916 rows=18.00 loops=1)
               Group Key: t.id_sala
               Buffers: shared hit=55
               ->  Sort  (cost=210.94..210.99 rows=20 width=8) (actual time=0.905..0.908 rows=20.00 loops=1)
                     Sort Key: t.id_sala
                     Sort Method: quicksort  Memory: 25kB
                     Buffers: shared hit=55
                     ->  Hash Join  (cost=144.25..210.51 rows=20 width=8) (actual time=0.487..0.895 rows=20.00 loops=1)
                           Hash Cond: (t.nro_solicitud = rt.nro_solicitud)
                           Buffers: shared hit=55
                           ->  Seq Scan on transferencia t  (cost=0.00..61.00 rows=2000 width=8) (actual time=0.012..0.349 rows=2000.00 loops=1)
                                 Filter: ((fecha_confeccion >= (CURRENT_DATE - '15 days'::interval)) AND (fecha_confeccion < (CURRENT_DATE + '1 day'::interval)))
                                 Buffers: shared hit=11
                           ->  Hash  (cost=144.00..144.00 rows=20 width=8) (actual time=0.432..0.432 rows=20.00 loops=1)
                                 Buckets: 1024  Batches: 1  Memory Usage: 9kB
                                 Buffers: shared hit=44
                                 ->  Seq Scan on renglon_transferencia rt  (cost=0.00..144.00 rows=20 width=8) (actual time=0.027..0.423 rows=20.00 loops=1)
                                       Filter: (cod_vino = 681)
                                       Rows Removed by Filter: 7980
                                       Buffers: shared hit=44
 Planning:
   Buffers: shared hit=142 read=6 dirtied=4
 Planning Time: 0.877 ms
 Execution Time: 1.041 ms
(30 rows)
```

S4 antes: Seq Scan sobre 8.000 renglones, con 7.980 descartados.

```
PS C:\Users\joseg\OneDrive\Documentos\3\Base de Datos\TRABAJO FINAL\trabajo_integrador_bd> Get-Content .\fase4\evidencias\jose\s4_despues_1.txt
                                                
```



```
bodegas_fase4_s12=# \o | tee /tmp/antonio/s12a_include_3.txt
bodegas_fase4_s12=# \i /tmp/antonio/s12a_explain.sql
-- S12-A (ver consultas.sql)
EXPLAIN (ANALYZE, BUFFERS)
SELECT c.id_cliente,
       c.nombre_razon_social
FROM cliente AS c
WHERE NOT EXISTS (
    SELECT 1
    FROM vino AS v
    JOIN bodega AS b ON b.cuit = v.cuit_bodega
    WHERE b.razon_social = 'Bodega DEMO Atuel 001'
      AND NOT EXISTS (
          SELECT 1
          FROM comprobante AS co
          JOIN renglon_comprobante AS rc ON rc.nro_comprobante = co.nro_comprobante
          WHERE co.id_cliente = c.id_cliente
            AND rc.cod_vino = v.cod_vino
      )
)
ORDER BY c.id_cliente;
Time: 366.765 ms
                                                
```



```
D:\FACULTAD\3º AÑO\BASE DE DATOS\tp-final-bd\trabajo_integrador_bd>type evidencias_fase4\lautaro\s1_despues_2.txt
´ïî                                                
```

5.3. S11-B – Diferencia atribuible a estadísticas, no al indice

En S11-B, el tiempo bajó de 47,448 a 28,259 ms después de crear un índice de expresión, pero el plan no usaba ese índice. Al quitarlo y actualizar las estadísticas sobre la misma expresión, se obtuvo un tiempo similar (28,657 ms). Concluimos que la mejora no provenía de leer el índice, así que también lo descartamos.

```
bodegas_fase4_toni=# \o | tee /tmp/antonio/s11b_estadisticas_3.txt
bodegas_fase4_toni=# \i /tmp/antonio/s11b_explain.sql
-- S11-B (ver consultas.sql)
EXPLAIN (ANALYZE, BUFFERS)
SELECT co.id_sala,
       date_trunc('month', co.fecha)::date AS mes,
       SUM(co.total) AS ventas_mes,
       SUM(SUM(co.total)) OVER (
           PARTITION BY co.id_sala
           ORDER BY date_trunc('month', co.fecha)
       ) AS ventas_acumuladas
FROM comprobante AS co
GROUP BY co.id_sala, date_trunc('month', co.fecha)
ORDER BY co.id_sala, mes;
bodegas_fase4_toni=#                                                                                                    QUERY PLAN
------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------
 Incremental Sort  (cost=4314.13..4328.66 rows=50 width=80) (actual time=27.181..28.578 rows=50.00 loops=1)
   Sort Key: id_sala, (((date_trunc('month'::text, fecha)))::date)
   Presorted Key: id_sala
   Full-sort Groups: 2  Sort Method: quicksort  Average Memory: 26kB  Peak Memory: 26kB
   Buffers: shared hit=1561
   ->  WindowAgg  (cost=4313.87..4326.41 rows=50 width=80) (actual time=27.094..28.561 rows=50.00 loops=1)
         Window: w1 AS (PARTITION BY id_sala ORDER BY (date_trunc('month'::text, fecha)))
         Storage: Memory  Maximum Storage: 17kB
         Buffers: shared hit=1561
         ->  Finalize GroupAggregate  (cost=4313.87..4325.16 rows=50 width=52) (actual time=27.080..28.507 rows=50.00 loops=1)
               Group Key: id_sala, (date_trunc('month'::text, fecha))
               Buffers: shared hit=1561
               ->  Gather Merge  (cost=4313.87..4323.56 rows=85 width=52) (actual time=27.070..28.448 rows=100.00 loops=1)
                     Workers Planned: 1
                     Workers Launched: 1
                     Buffers: shared hit=1561
                     ->  Sort  (cost=3313.86..3313.99 rows=50 width=52) (actual time=24.759..24.763 rows=50.00 loops=2)
                           Sort Key: id_sala, (date_trunc('month'::text, fecha))
                           Sort Method: quicksort  Memory: 30kB
                           Buffers: shared hit=1561
                           Worker 0:  Sort Method: quicksort  Memory: 30kB
                           ->  Partial HashAggregate  (cost=3311.70..3312.45 rows=50 width=52) (actual time=24.696..24.706 rows=50.00 loops=2)
                                 Group Key: id_sala, date_trunc('month'::text, fecha)
                                 Batches: 1  Memory Usage: 40kB
                                 Buffers: shared hit=1547
                                 Worker 0:  Batches: 1  Memory Usage: 40kB
                                 ->  Parallel Seq Scan on comprobante co  (cost=0.00..2649.94 rows=88235 width=28) (actual time=0.012..10.439 rows=75000.00 loops=2)
                                       Buffers: shared hit=1547
 Planning Time: 0.169 ms
 Execution Time: 28.657 ms
(30 rows)

\o
bodegas_fase4_toni=# \o
```

S11-B: las estadísticas de la expresión reproducen el plan sin recorrer el índice.

5.4. S5 – Indice parcial con conjunto vacío

El índice parcial idx_transferencia_pendiente_sala se creó para las transferencias cuyo estado es distinto de 'procesado'. PostgreSQL lo utilizó y el tiempo pasó de 0,311 a 0,198 ms. Sin embargo, las 2.000 transferencias del dump ya estaban procesadas, por lo que no había pendientes. Por ese motivo lo conservamos como índice parcial válido, pero no contamos S5 entre las tres mejoras principales.

```
PS C:\Users\joseg\OneDrive\Documentos\3\Base de Datos\TRABAJO FINAL\trabajo_integrador_bd> Get-Content .\fase4\evidencias\jose\s5_despues_1.txt
                                                                                                    QUERY PLAN
------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=12.01..12.13 rows=50 width=31) (actual time=0.103..0.105 rows=50.00 loops=1)
   Sort Key: (count(t.nro_solicitud)), s.id_sala
   Sort Method: quicksort  Memory: 27kB
   Buffers: shared hit=8
   ->  HashAggregate  (cost=10.10..10.60 rows=50 width=31) (actual time=0.066..0.072 rows=50.00 loops=1)
         Group Key: s.id_sala
         Batches: 1  Memory Usage: 32kB
         Buffers: shared hit=2
         ->  Hash Left Join  (cost=8.15..9.85 rows=50 width=27) (actual time=0.047..0.055 rows=50.00 loops=1)
               Hash Cond: (s.id_sala = t.id_sala)
               Buffers: shared hit=2
               ->  Seq Scan on sala_venta s  (cost=0.00..1.50 rows=50 width=23) (actual time=0.018..0.020 rows=50.00 loops=1)
                     Buffers: shared hit=1
               ->  Hash  (cost=8.14..8.14 rows=1 width=8) (actual time=0.012..0.012 rows=0.00 loops=1)
                     Buckets: 1024  Batches: 1  Memory Usage: 8kB
                     Buffers: shared hit=1
                     ->  Index Scan using idx_transferencia_pendiente_sala on transferencia t  (cost=0.12..8.14 rows=1 width=8) (actual time=0.012..0.012 rows=0.00 loops=1)
                           Index Searches: 1
                           Buffers: shared hit=1
 Planning:
   Buffers: shared hit=162 read=1
 Planning Time: 0.772 ms
 Execution Time: 0.195 ms
(23 rows)
```

S5: utilización del índice parcial con cero filas pendientes en el dataset.

6. Costo de escritura: Prueba con 10.000 comprobantes

Fabricio comparó la inserción de 10.000 cabeceras de comprobantes en dos copias de la misma base: una con los índices originales y otra con dos índices adicionales sobre comprobante. Mantuvo activas las restricciones y los triggers. Hizo tres pruebas por escenario, utilizando BEGIN y ROLLBACK para no dejar registros insertados.

Repetición	Sin índices nuevos (ms)	Con dos índices nuevos (ms)
1	138,658	236,857
2	151,735	142,877
3	123,761	143,219
Mediana	138,658	143,219

La mediana pasó de 138,658 ms sin índices adicionales a 143,219 ms con ellos. Eso representa un sobrecosto observado del 3,29 %. La diferencia fue pequeña y los tiempos variaron entre repeticiones; por eso no se puede afirmar que agregar índices siempre aumente el tiempo en ese porcentaje. Esta prueba mide cabeceras de comprobantes, no ventas completas.

```
BEGIN
                                                
```

Los resultados confirman que PostgreSQL sigue utilizando los índices de S3, S4 y S12-A cuando están los cinco instalados. Estas mediciones solo comprueban el funcionamiento conjunto: las comparaciones antes y después son las de las pruebas individuales.

```
                                                
```

8. Limitaciones de los resultados

Las pruebas se hicieron en equipos diferentes, por lo que los tiempos pueden variar según la computadora y el estado de la base. Por eso usamos tres repeticiones y revisamos los planes, además del tiempo. En S8 y S11-B comprobamos que una consulta puede tardar menos sin que la causa sea el índice probado.

También encontramos dos limitaciones en los datos: todos los comprobantes comerciales tienen la misma fecha y no hay transferencias pendientes. Además, S12-A no devolvió clientes para la bodega elegida. Estos resultados sirven para evaluar las consultas probadas, pero no garantizan el mismo comportamiento con otros datos.

9. Conclusión técnica

En la Fase 4 creamos y probamos cinco índices de los tipos pedidos: compuesto, parcial, sobre clave foránea y especial con INCLUDE. Comprobamos mejoras en tres consultas oficiales: S3 (64,26 %), S4 (30,93 %) y S12-A (99,65 %). También explicamos por qué descartamos algunos índices y mostramos casos donde PostgreSQL decidió no utilizarlos.

La prueba final confirmó que los cinco índices estaban creados y que los correspondientes a S3, S4 y S12-A seguían utilizándose. En el ensayo de inserción observamos un 3,29 % más de tiempo al mantener dos índices adicionales. Esto nos mostró que un índice puede ayudar en las búsquedas, pero también tiene un costo al insertar datos; por eso es importante probarlo antes de incorporarlo.

10. Fuentes y ubicación de evidencias

Nos basamos en la consigna oficial «TPI Bases de Datos — Bodegas del Atuel», Etapa 4 (páginas 14 y 15), y en las consultas de la sección 8. El archivo de índices definitivo es db/migrations/008_indices.sql y los datos originales proceden del respaldo full de la Fase 3.

Los archivos originales quedaron organizados en fase4/evidencias/lautaro/, fase4/evidencias/antonio/, fase4/evidencias/jose/, fase4/evidencias/fabricio/ y fase4/evidencias/integracion/. Allí se conservan las consultas, los reportes, las capturas y las tres repeticiones de cada prueba. En el anexo incluimos planes completos seleccionados que respaldan las mejoras principales y las decisiones de descarte.

Anexo – Planes completos de ejecución

Se incluyen los planes completos de las tres consultas principales, más ejemplos de índices descartados o no utilizados. Son los registros originales de PostgreSQL; las demás repeticiones se conservan en las carpetas de evidencias del equipo. Las capturas ya aparecen en el desarrollo y no se vuelven a insertar aquí.

A1. S3 — antes del índice

Lautaro · 35,317 ms · Archivo: s3_antes_2.txt

```
GroupAggregate  (cost=5876.29..6220.62 rows=3 width=17) (actual time=31.424..35.095 rows=3.00 loops=1)
  Group Key: c.tipo_cliente
  Buffers: shared hit=1919
  ->  Gather Merge  (cost=5876.29..6206.12 rows=2894 width=13) (actual time=31.111..34.738 rows=3000.00 loops=1)
        Workers Planned: 1
        Workers Launched: 1
        Buffers: shared hit=1919
        ->  Sort  (cost=4876.28..4880.53 rows=1702 width=13) (actual time=10.027..10.127 rows=1500.00 loops=2)
              Sort Key: c.tipo_cliente, c.id_cliente
              Sort Method: quicksort  Memory: 198kB
              Buffers: shared hit=1919
              Worker 0:  Sort Method: quicksort  Memory: 25kB
              ->  Hash Join  (cost=794.00..4784.94 rows=1702 width=13) (actual time=2.306..9.052 rows=1500.00
    loops=2)
                    Hash Cond: (co.id_cliente = c.id_cliente)
                    Buffers: shared hit=1904
                    ->  Parallel Seq Scan on comprobante co  (cost=0.00..3986.47 rows=1702 width=4) (actual
    time=0.007..6.176 rows=1500.00 loops=2)
                          Filter: ((id_sala = 29) AND (fecha >= (CURRENT_DATE - '7 days'::interval)) AND (fecha <
    (CURRENT_DATE + '1 day'::interval)))
                          Rows Removed by Filter: 73500
                          Buffers: shared hit=1560
                    ->  Hash  (cost=544.00..544.00 rows=20000 width=13) (actual time=4.535..4.536 rows=20000.00
    loops=1)
                          Buckets: 32768  Batches: 1  Memory Usage: 1194kB
                          Buffers: shared hit=344
                          ->  Seq Scan on cliente c  (cost=0.00..544.00 rows=20000 width=13) (actual
    time=0.012..2.560 rows=20000.00 loops=1)
                                Buffers: shared hit=344
Planning:
  Buffers: shared hit=168
Planning Time: 3.708 ms
Execution Time: 35.317 ms
(28 rows)
```

A2. S3 — después del índice

Lautaro · 12,623 ms · Archivo: s3_despues_1.txt

```
GroupAggregate  (cost=2719.46..2741.19 rows=3 width=17) (actual time=12.173..12.398 rows=3.00 loops=1)
  Group Key: c.tipo_cliente
  Buffers: shared hit=1897 read=14
  ->  Sort  (cost=2719.46..2726.69 rows=2894 width=13) (actual time=12.044..12.138 rows=3000.00 loops=1)
        Sort Key: c.tipo_cliente, c.id_cliente
        Sort Method: quicksort  Memory: 198kB
        Buffers: shared hit=1897 read=14
        ->  Hash Join  (cost=879.33..2553.07 rows=2894 width=13) (actual time=5.417..10.340 rows=3000.00 loops=1)
              Hash Cond: (co.id_cliente = c.id_cliente)
              Buffers: shared hit=1891 read=14
              ->  Bitmap Heap Scan on comprobante co  (cost=85.33..1751.47 rows=2894 width=4) (actual
    time=0.610..4.856 rows=3000.00 loops=1)
                    Recheck Cond: ((id_sala = 29) AND (fecha >= (CURRENT_DATE - '7 days'::interval)) AND (fecha <
    (CURRENT_DATE + '1 day'::interval)))
                    Heap Blocks: exact=1547
                    Buffers: shared hit=1547 read=14
                    ->  Bitmap Index Scan on idx_comprobante_sala_fecha  (cost=0.00..84.61 rows=2894 width=0)
    (actual time=0.466..0.467 rows=3000.00 loops=1)
                          Index Cond: ((id_sala = 29) AND (fecha >= (CURRENT_DATE - '7 days'::interval)) AND (fecha
    < (CURRENT_DATE + '1 day'::interval)))
                          Index Searches: 1
                          Buffers: shared read=14
              ->  Hash  (cost=544.00..544.00 rows=20000 width=13) (actual time=4.679..4.679 rows=20000.00 loops=1)
                    Buckets: 32768  Batches: 1  Memory Usage: 1194kB
                    Buffers: shared hit=344
                    ->  Seq Scan on cliente c  (cost=0.00..544.00 rows=20000 width=13) (actual time=0.023..2.615
    rows=20000.00 loops=1)
                          Buffers: shared hit=344
Planning:
  Buffers: shared hit=200 read=1
Planning Time: 4.802 ms
Execution Time: 12.623 ms
(27 rows)
```

A3. S4 — antes del índice

José · 1,041 ms · Archivo: s4_antes_1.txt

```
Limit  (cost=211.63..211.64 rows=5 width=12) (actual time=0.943..0.946 rows=5.00 loops=1)
  Buffers: shared hit=61
  ->  Sort  (cost=211.63..211.68 rows=20 width=12) (actual time=0.942..0.944 rows=5.00 loops=1)
        Sort Key: (sum(rt.cantidad)) DESC, t.id_sala
        Sort Method: top-N heapsort  Memory: 25kB
        Buffers: shared hit=61
        ->  GroupAggregate  (cost=210.94..211.29 rows=20 width=12) (actual time=0.909..0.916 rows=18.00 loops=1)
              Group Key: t.id_sala
              Buffers: shared hit=55
              ->  Sort  (cost=210.94..210.99 rows=20 width=8) (actual time=0.905..0.908 rows=20.00 loops=1)
                    Sort Key: t.id_sala
                    Sort Method: quicksort  Memory: 25kB
                    Buffers: shared hit=55
                    ->  Hash Join  (cost=144.25..210.51 rows=20 width=8) (actual time=0.487..0.895 rows=20.00
    loops=1)
                          Hash Cond: (t.nro_solicitud = rt.nro_solicitud)
                          Buffers: shared hit=55
                          ->  Seq Scan on transferencia t  (cost=0.00..61.00 rows=2000 width=8) (actual
    time=0.012..0.349 rows=2000.00 loops=1)
                                Filter: ((fecha_confeccion >= (CURRENT_DATE - '15 days'::interval)) AND
    (fecha_confeccion < (CURRENT_DATE + '1 day'::interval)))
                                Buffers: shared hit=11
                          ->  Hash  (cost=144.00..144.00 rows=20 width=8) (actual time=0.432..0.432 rows=20.00
    loops=1)
                                Buckets: 1024  Batches: 1  Memory Usage: 9kB
                                Buffers: shared hit=44
                                ->  Seq Scan on renglon_transferencia rt  (cost=0.00..144.00 rows=20 width=8)
    (actual time=0.027..0.423 rows=20.00 loops=1)
                                      Filter: (cod_vino = 681)
                                      Rows Removed by Filter: 7980
                                      Buffers: shared hit=44
Planning:
  Buffers: shared hit=142 read=6 dirtied=4
Planning Time: 0.877 ms
Execution Time: 1.041 ms
(30 rows)
```

A4. S4 — después del índice

José · 0,719 ms · Archivo: s4_despues_1.txt

```
Limit  (cost=108.61..108.62 rows=5 width=12) (actual time=0.624..0.625 rows=5.00 loops=1)
  Buffers: shared hit=33 read=2
  ->  Sort  (cost=108.61..108.66 rows=20 width=12) (actual time=0.622..0.623 rows=5.00 loops=1)
        Sort Key: (sum(rt.cantidad)) DESC, t.id_sala
        Sort Method: top-N heapsort  Memory: 25kB
        Buffers: shared hit=33 read=2
        ->  GroupAggregate  (cost=107.93..108.28 rows=20 width=12) (actual time=0.586..0.591 rows=18.00 loops=1)
              Group Key: t.id_sala
              Buffers: shared hit=27 read=2
              ->  Sort  (cost=107.93..107.98 rows=20 width=8) (actual time=0.582..0.584 rows=20.00 loops=1)
                    Sort Key: t.id_sala
                    Sort Method: quicksort  Memory: 25kB
                    Buffers: shared hit=27 read=2
                    ->  Hash Join  (cost=41.24..107.50 rows=20 width=8) (actual time=0.170..0.571 rows=20.00
    loops=1)
                          Hash Cond: (t.nro_solicitud = rt.nro_solicitud)
                          Buffers: shared hit=27 read=2
                          ->  Seq Scan on transferencia t  (cost=0.00..61.00 rows=2000 width=8) (actual
    time=0.014..0.347 rows=2000.00 loops=1)
                                Filter: ((fecha_confeccion >= (CURRENT_DATE - '15 days'::interval)) AND
    (fecha_confeccion < (CURRENT_DATE + '1 day'::interval)))
                                Buffers: shared hit=11
                          ->  Hash  (cost=40.99..40.99 rows=20 width=8) (actual time=0.120..0.120 rows=20.00
    loops=1)
                                Buckets: 1024  Batches: 1  Memory Usage: 9kB
                                Buffers: shared hit=16 read=2
                                ->  Bitmap Heap Scan on renglon_transferencia rt  (cost=4.44..40.99 rows=20
    width=8) (actual time=0.054..0.110 rows=20.00 loops=1)
                                      Recheck Cond: (cod_vino = 681)
                                      Heap Blocks: exact=16
                                      Buffers: shared hit=16 read=2
                                      ->  Bitmap Index Scan on idx_rt_cod_vino_nro  (cost=0.00..4.43 rows=20
    width=0) (actual time=0.043..0.043 rows=20.00 loops=1)
                                            Index Cond: (cod_vino = 681)
                                            Index Searches: 1
                                            Buffers: shared read=2
Planning:
  Buffers: shared hit=161 read=1
Planning Time: 0.894 ms
Execution Time: 0.719 ms
(34 rows)
```

A5. S12-A — antes del índice

Antonio · 104.823,719 ms · Archivo: s12a_antes_2.txt

```
Nested Loop Anti Join  (cost=1.67..930883648.17 rows=10000 width=24) (actual time=104746.870..104748.964 rows=0.00
    loops=1)
  Join Filter: (NOT EXISTS(SubPlan 1))
  Rows Removed by Join Filter: 370
  Buffers: shared hit=31674202
  ->  Index Scan using pk_cliente on cliente c  (cost=0.29..868.30 rows=20000 width=24) (actual time=1.236..230.594
    rows=20000.00 loops=1)
        Index Searches: 1
        Buffers: shared hit=424
  ->  Materialize  (cost=1.39..20.26 rows=25 width=4) (actual time=0.034..0.034 rows=1.02 loops=20000)
        Storage: Memory  Maximum Storage: 17kB
        Buffers: shared hit=3
        ->  Hash Join  (cost=1.39..20.13 rows=25 width=4) (actual time=669.498..669.598 rows=3.00 loops=1)
              Hash Cond: ((v.cuit_bodega)::bpchar = (b.cuit)::bpchar)
              Buffers: shared hit=3
              ->  Seq Scan on vino v  (cost=0.00..16.50 rows=750 width=16) (actual time=0.480..0.487 rows=59.00
    loops=1)
                    Buffers: shared hit=2
              ->  Hash  (cost=1.38..1.38 rows=1 width=12) (actual time=668.164..668.194 rows=1.00 loops=1)
                    Buckets: 1024  Batches: 1  Memory Usage: 9kB
                    Buffers: shared hit=1
                    ->  Seq Scan on bodega b  (cost=0.00..1.38 rows=1 width=12) (actual time=667.690..667.696
    rows=1.00 loops=1)
                          Filter: ((razon_social)::text = 'Bodega DEMO Atuel 001'::text)
                          Rows Removed by Filter: 29
                          Buffers: shared hit=1
  SubPlan 1
    ->  Nested Loop  (cost=0.42..3457.55 rows=1 width=0) (actual time=5.094..5.094 rows=0.02 loops=20370)
          Buffers: shared hit=31673775
          ->  Seq Scan on comprobante co  (cost=0.00..3422.00 rows=8 width=4) (actual time=0.676..5.029 rows=7.45
    loops=20370)
                Filter: (id_cliente = c.id_cliente)
                Rows Removed by Filter: 148592
                Buffers: shared hit=31218623
          ->  Index Only Scan using pk_renglon_comprobante on renglon_comprobante rc  (cost=0.42..4.44 rows=1
    width=4) (actual time=0.006..0.006 rows=0.00 loops=151717)
                Index Cond: ((nro_comprobante = co.nro_comprobante) AND (cod_vino = v.cod_vino))
                Heap Fetches: 0
                Index Searches: 151717
                Buffers: shared hit=455152
Planning:
  Buffers: shared hit=36
Planning Time: 64.474 ms
JIT:
  Functions: 23
  Options: Inlining true, Optimization true, Expressions true, Deforming true
  Timing: Generation 62.603 ms (Deform 10.695 ms), Inlining 176.734 ms, Optimization 259.356 ms, Emission 230.802
    ms, Total 729.495 ms
Execution Time: 104823.719 ms
(42 rows)
```

A6. S12-A — después del INCLUDE

Antonio · 365,214 ms · Archivo: s12a_include_3.txt

```
Nested Loop Anti Join  (cost=1.67..10803648.17 rows=10000 width=24) (actual time=363.798..363.803 rows=0.00
     loops=1)
  Join Filter: (NOT EXISTS(SubPlan 1))
  Rows Removed by Join Filter: 370
  Buffers: shared hit=517047
  ->  Index Scan using pk_cliente on cliente c  (cost=0.29..868.30 rows=20000 width=24) (actual time=0.038..4.261
     rows=20000.00 loops=1)
        Index Searches: 1
        Buffers: shared hit=424
  ->  Materialize  (cost=1.39..20.26 rows=25 width=4) (actual time=0.008..0.008 rows=1.02 loops=20000)
        Storage: Memory  Maximum Storage: 17kB
        Buffers: shared hit=3
        ->  Hash Join  (cost=1.39..20.13 rows=25 width=4) (actual time=149.908..149.928 rows=3.00 loops=1)
              Hash Cond: ((v.cuit_bodega)::bpchar = (b.cuit)::bpchar)
              Buffers: shared hit=3
              ->  Seq Scan on vino v  (cost=0.00..16.50 rows=750 width=16) (actual time=0.011..0.016 rows=59.00
     loops=1)
                    Buffers: shared hit=2
              ->  Hash  (cost=1.38..1.38 rows=1 width=12) (actual time=149.868..149.870 rows=1.00 loops=1)
                    Buckets: 1024  Batches: 1  Memory Usage: 9kB
                    Buffers: shared hit=1
                    ->  Seq Scan on bodega b  (cost=0.00..1.38 rows=1 width=12) (actual time=149.854..149.858
     rows=1.00 loops=1)
                          Filter: ((razon_social)::text = 'Bodega DEMO Atuel 001'::text)
                          Rows Removed by Filter: 29
                          Buffers: shared hit=1
  SubPlan 1
    ->  Nested Loop  (cost=0.84..40.11 rows=1 width=0) (actual time=0.010..0.010 rows=0.02 loops=20370)
          Buffers: shared hit=516620
          ->  Index Only Scan using idx_comprobante_id_cliente_inc on comprobante co  (cost=0.42..4.56 rows=8
     width=4) (actual time=0.001..0.001 rows=7.45 loops=20370)
                Index Cond: (id_cliente = c.id_cliente)
                Heap Fetches: 0
                Index Searches: 20370
                Buffers: shared hit=61468
          ->  Index Only Scan using pk_renglon_comprobante on renglon_comprobante rc  (cost=0.42..4.44 rows=1
     width=4) (actual time=0.001..0.001 rows=0.00 loops=151717)
                Index Cond: ((nro_comprobante = co.nro_comprobante) AND (cod_vino = v.cod_vino))
                Heap Fetches: 0
                Index Searches: 151717
                Buffers: shared hit=455152
Planning:
  Buffers: shared hit=36
Planning Time: 0.697 ms
JIT:
  Functions: 23
  Options: Inlining true, Optimization true, Expressions true, Deforming true
  Timing: Generation 1.212 ms (Deform 0.416 ms), Inlining 40.961 ms, Optimization 71.336 ms, Emission 37.561 ms,
     Total 151.069 ms
Execution Time: 365.214 ms
(43 rows)
```

A7. S1 — índice existente no utilizado

Lautaro · 86,980 ms · Archivo: s1_despues_2.txt

```
Limit  (cost=9090.53..9090.55 rows=10 width=30) (actual time=82.467..86.847 rows=10.00 loops=1)
  Buffers: shared hit=3734 read=414
  ->  Sort  (cost=9090.53..9092.40 rows=750 width=30) (actual time=82.465..86.845 rows=10.00 loops=1)
        Sort Key: (sum(rc.cantidad)) DESC, v.cod_vino
        Sort Method: top-N heapsort  Memory: 26kB
        Buffers: shared hit=3734 read=414
        ->  Finalize GroupAggregate  (cost=8848.18..9074.32 rows=750 width=30) (actual time=81.882..86.747
     rows=750.00 loops=1)
              Group Key: v.cod_vino
              Buffers: shared hit=3731 read=414
              ->  Gather Merge  (cost=8848.18..9057.82 rows=1800 width=30) (actual time=81.874..86.496 rows=2250.00
     loops=1)
                    Workers Planned: 2
                    Workers Launched: 2
                    Buffers: shared hit=3731 read=414
                    ->  Sort  (cost=7848.16..7850.03 rows=750 width=30) (actual time=58.319..58.352 rows=750.00
     loops=3)
                          Sort Key: v.cod_vino
                          Sort Method: quicksort  Memory: 60kB
                          Buffers: shared hit=3731 read=414
                          Worker 0:  Sort Method: quicksort  Memory: 60kB
                          Worker 1:  Sort Method: quicksort  Memory: 60kB
                          ->  Partial HashAggregate  (cost=7804.84..7812.34 rows=750 width=30) (actual
     time=58.072..58.154 rows=750.00 loops=3)
                                Group Key: v.cod_vino
                                Batches: 1  Memory Usage: 105kB
                                Buffers: shared hit=3717 read=414
                                Worker 0:  Batches: 1  Memory Usage: 105kB
                                Worker 1:  Batches: 1  Memory Usage: 105kB
                                ->  Hash Join  (cost=25.88..6763.18 rows=208333 width=26) (actual
     time=0.522..37.852 rows=166666.67 loops=3)
                                      Hash Cond: (rc.cod_vino = v.cod_vino)
                                      Buffers: shared hit=3717 read=414
                                      ->  Parallel Seq Scan on renglon_comprobante rc  (cost=0.00..6187.33
     rows=208333 width=8) (actual time=0.044..8.163 rows=166666.67 loops=3)
                                            Buffers: shared hit=3690 read=414
                                      ->  Hash  (cost=16.50..16.50 rows=750 width=22) (actual time=0.456..0.457
     rows=750.00 loops=3)
                                            Buckets: 1024  Batches: 1  Memory Usage: 48kB
                                            Buffers: shared hit=27
                                            ->  Seq Scan on vino v  (cost=0.00..16.50 rows=750 width=22) (actual
     time=0.311..0.382 rows=750.00 loops=3)
                                                  Buffers: shared hit=27
Planning:
  Buffers: shared hit=195
Planning Time: 4.006 ms
Execution Time: 86.980 ms
(39 rows)
```

A8. S8 — índice BRIN descartado

Antonio · 73,998 ms · Archivo: s8_despues_brin_2.txt

```
Sort  (cost=5461.70..5461.83 rows=50 width=55) (actual time=71.787..73.879 rows=50.00 loops=1)
  Sort Key: (sum(co.total)) DESC
  Sort Method: quicksort  Memory: 27kB
  Buffers: shared hit=1556
  ->  Finalize GroupAggregate  (cost=5449.34..5460.29 rows=50 width=55) (actual time=71.698..73.857 rows=50.00
     loops=1)
        Group Key: s.id_sala
        Buffers: shared hit=1556
        ->  Gather Merge  (cost=5449.34..5459.03 rows=85 width=55) (actual time=71.689..73.794 rows=100.00 loops=1)
              Workers Planned: 1
              Workers Launched: 1
              Buffers: shared hit=1556
              ->  Sort  (cost=4449.33..4449.46 rows=50 width=55) (actual time=67.522..67.527 rows=50.00 loops=2)
                    Sort Key: s.id_sala
                    Sort Method: quicksort  Memory: 30kB
                    Buffers: shared hit=1556
                    Worker 0:  Sort Method: quicksort  Memory: 30kB
                    ->  Partial HashAggregate  (cost=4447.29..4447.92 rows=50 width=55) (actual time=67.485..67.499
     rows=50.00 loops=2)
                          Group Key: s.id_sala
                          Batches: 1  Memory Usage: 40kB
                          Buffers: shared hit=1549
                          Worker 0:  Batches: 1  Memory Usage: 40kB
                          ->  Hash Join  (cost=2.12..4006.20 rows=88218 width=31) (actual time=0.272..43.395
     rows=75000.00 loops=2)
                                Hash Cond: (co.id_sala = s.id_sala)
                                Buffers: shared hit=1549
                                ->  Parallel Seq Scan on comprobante co  (cost=0.00..3752.88 rows=88218 width=12)
     (actual time=0.208..24.049 rows=75000.00 loops=2)
                                      Filter: ((fecha >= (CURRENT_DATE - '3 mons'::interval)) AND (fecha <
     (CURRENT_DATE + '1 day'::interval)))
                                      Buffers: shared hit=1547
                                ->  Hash  (cost=1.50..1.50 rows=50 width=23) (actual time=0.044..0.045 rows=50.00
     loops=2)
                                      Buckets: 1024  Batches: 1  Memory Usage: 11kB
                                      Buffers: shared hit=2
                                      ->  Seq Scan on sala_venta s  (cost=0.00..1.50 rows=50 width=23) (actual
     time=0.025..0.031 rows=50.00 loops=2)
                                            Buffers: shared hit=2
Planning:
  Buffers: shared hit=5
Planning Time: 0.392 ms
Execution Time: 73.998 ms
(36 rows)
```

A9. S11-B — índice de expresión descartado

Antonio · 28,259 ms · Archivo: s11b_despues_expr_2.txt

```
Incremental Sort  (cost=4314.13..4328.66 rows=50 width=80) (actual time=26.654..28.110 rows=50.00 loops=1)
  Sort Key: id_sala, (((date_trunc('month'::text, fecha)))::date)
  Presorted Key: id_sala
  Full-sort Groups: 2  Sort Method: quicksort  Average Memory: 26kB  Peak Memory: 26kB
  Buffers: shared hit=1561
  ->  WindowAgg  (cost=4313.87..4326.41 rows=50 width=80) (actual time=26.596..28.097 rows=50.00 loops=1)
        Window: w1 AS (PARTITION BY id_sala ORDER BY (date_trunc('month'::text, fecha)))
        Storage: Memory  Maximum Storage: 17kB
        Buffers: shared hit=1561
        ->  Finalize GroupAggregate  (cost=4313.87..4325.16 rows=50 width=52) (actual time=26.581..28.057
     rows=50.00 loops=1)
              Group Key: id_sala, (date_trunc('month'::text, fecha))
              Buffers: shared hit=1561
              ->  Gather Merge  (cost=4313.87..4323.56 rows=85 width=52) (actual time=26.548..27.991 rows=100.00
     loops=1)
                    Workers Planned: 1
                    Workers Launched: 1
                    Buffers: shared hit=1561
                    ->  Sort  (cost=3313.86..3313.99 rows=50 width=52) (actual time=23.901..23.903 rows=50.00
     loops=2)
                          Sort Key: id_sala, (date_trunc('month'::text, fecha))
                          Sort Method: quicksort  Memory: 30kB
                          Buffers: shared hit=1561
                          Worker 0:  Sort Method: quicksort  Memory: 30kB
                          ->  Partial HashAggregate  (cost=3311.70..3312.45 rows=50 width=52) (actual
     time=23.866..23.876 rows=50.00 loops=2)
                                Group Key: id_sala, date_trunc('month'::text, fecha)
                                Batches: 1  Memory Usage: 40kB
                                Buffers: shared hit=1547
                                Worker 0:  Batches: 1  Memory Usage: 40kB
                                ->  Parallel Seq Scan on comprobante co  (cost=0.00..2649.94 rows=88235 width=28)
     (actual time=0.011..9.990 rows=75000.00 loops=2)
                                      Buffers: shared hit=1547
Planning Time: 0.143 ms
Execution Time: 28.259 ms
(30 rows)
```

INFORME FASE 5

En esta etapa se resolvieron consultas SQL del sistema vitivinícola a partir del modelo y del conjunto de datos cargado en la Fase 3. La entrega oficial está organizada en cuatro scripts versionados en **consultas/**.

1. Objetivo y alcance

Aplicar instrucciones de modificación y consultas de análisis sobre PostgreSQL, utilizando JOIN, subconsultas, CTE, funciones de ventana y controles transaccionales. La revisión del código integrado permite verificar la presencia de las **30 consignas** solicitadas.

2. Organización de la entrega

Archivo	Consignas verificadas	Control implementado
update.sql	U1–U6 (6)	6 bloques BEGIN / ROLLBACK
insert.sql	I1–I6 (6)	6 bloques BEGIN / ROLLBACK
delete.sql	D1–D6 (6)	6 bloques BEGIN / ROLLBACK
select.sql	S1–S12 (12)	Consultas analíticas y planes EXPLAIN seleccionados

3. Casos representativos

Caso	Qué se resolvió	Evidencia en el SQL
U3	Incremento de precio de vinos de una bodega.	Control previo y posterior de RN-13; ROLLBACK.
I3	Comprobante comercial con tres renglones.	Controles de totales, descuento, stock y reversión.
S11	Ranking y acumulado mensual.	Funciones de ventana sobre ventas.
S12	Clientes que compraron todos los vinos de una bodega y clientes que nunca compraron tintos.	Doble NOT EXISTS y planes para ambas partes.

4. Evidencia y conclusión técnica

Los archivos integrados incluyen consultas comentadas y, para las 18 consignas U/I/D, transacciones con **BEGIN y ROLLBACK**. Se localizaron cuatro sentencias **EXPLAIN (ANALYZE, BUFFERS)** en **select.sql**. Las capturas y salidas individuales citadas por los comentarios del código no estaban publicadas en la rama revisada, por lo que **no se presentan métricas nuevas ni se afirma aquí una ejecución conjunta verificada**.

Fuente principal: consultas/update.sql, insert.sql, delete.sql y select.sql.

INFORME FASE 6

Se implementó una API REST en **FastAPI** que permite consultar y operar sobre la base PostgreSQL. La aplicación utiliza **Psycopg 3**, un pool de conexiones y consultas SQL explícitas parametrizadas, sin ORM. Las reglas de integridad y las modificaciones de existencias siguen a cargo de PostgreSQL.

1. Implementación y arquitectura

El código se organiza en **routers** (HTTP), **schemas** (validación), **repositories** (SQL) y **services** (transacciones). La conexión se configura por variables de entorno; el rol de trabajo es **bodega_api**, sin privilegios de superusuario. El manejador central traduce errores SQLSTATE a respuestas HTTP.

Módulo	Endpoints funcionales	Operaciones principales
Vinos	5	Listado, detalle, alta, edición y baja
Comprobantes	2	Consulta y registro de ventas
Ingresos	5	Alta, renglones, terminar, procesar y consultar
Transferencias	5	Alta, renglones, terminar, procesar y consultar
Clientes	5	Listado, detalle, alta, edición y baja
Reportes	5	Ranking, salas, faltantes, transportistas y trazabilidad
Total	**27**	Más GET /salud (comprobación de BD)

2. Ejecución reproducible con Docker

La composición **compose.fase6.yml** integra PostgreSQL y la API. La inicialización automática, cuando el volumen está vacío, restaura el respaldo, prepara roles y aplica los cinco índices de Fase 4. Se probó en un entorno Docker independiente el **05/11/2026**, sin modificar la base principal.

```
Respaldo SHA-256: OK    |   Restauracion: OK
27 tablas: OK           |   5 indices: OK
Salud: HTTP 200         |   Consultas HTTP: 11/11
Vinos: 750              |   Clientes: 20000
INICIALIZACION COMPLETA: OK  |  Codigo de salida: 0
```

Extracto fiel de fase6/evidencias/valentino/reconstruccion_docker.txt. Evidencia de reconstrucción, no captura recreada.

El registro del mismo día documenta también el arranque automático con volumen nuevo: PostgreSQL healthy, API conectada y 27 endpoints detectados.

FASE 6 · PRUEBAS Y RESULTADOS

3. Transacciones, seguridad y pruebas

Prueba	Resultado respaldado	Archivo original
RN-09 y rollback	Venta con segundo renglón sin stock: HTTP 409, P0001; sin cabeceras, renglones ni stock alterados.	rollback_venta_sin_stock.txt
Inyección SQL	Filtro malicioso en GET /vinos: 0 resultados; consulta normal y posterior: 750 vinos.	inyeccion_sql_fallida.txt
Pytest	Ejecución de módulo de Fabricio: 31 passed, 1 warning.	pruebas_finales.txt
Suite documentada	El README de API informa 66 pruebas automatizadas distribuidas entre los cinco módulos.	api/README.md

En la prueba transaccional, el error **RN-09** devolvió HTTP 409, y el control posterior sobre PostgreSQL comprobó que no persistieran registros parciales. La protección frente a inyección se verificó para el filtro probado; **no constituye una auditoría completa de toda la API**.

4. Evidencias seleccionadas

```
HTTP: 409       |  SQLSTATE: P0001
Regla de negocio: RN-09
Cantidad de comprobantes: sin cambios
Cantidad de renglones: sin cambios
Stock de ambos vinos: sin cambios
```

Fragmento del registro de rollback real; base de ensayo separada. No es una ejecución nueva.

```
Consulta normal: HTTP 200      |  Total: 750
Consulta con inyeccion: HTTP 200 |  Total: 0
Consulta posterior: HTTP 200   |  Total: 750
```

Fragmento del control de inyección SQL del filtro bodega en GET /vinos.

5. Conclusión y ubicación de evidencias

La API quedó organizada en módulos, conectada a PostgreSQL mediante un rol de privilegios limitados y validada con pruebas de integración y reconstrucción Docker. Las salidas seleccionadas acreditan los casos concretos documentados, sin extrapolar su resultado a todas las rutas.

Fuentes: api/README.md, api/app/, api/tests/, compose.fase6.yml y fase6/evidencias/{valentino,fabricio}/. La captura de inyección SQL también está versionada en **fase6/evidencias/valentino/inyeccion_sql_fallida.png**; no se copia porque su archivo de imagen no estuvo disponible para verificación visual en esta compilación.

INFORME FASE 7

Se desarrolló una interfaz web para gestionar información vitivinícola desde el navegador. El frontend utiliza **React, TypeScript, Vite y React Router** y consume la API FastAPI de Fase 6; las reglas de stock, descuentos y estados permanecen en PostgreSQL.

1. Objetivo y alcance

Integrar las pantallas de catálogo, vinos, ventas, transferencias y reportes con datos reales de la base, reutilizando un cliente HTTP que centraliza las solicitudes **/api**, los mensajes de error y las validaciones básicas de formulario. La referencia para esta fase es **integracion/fase7**, anterior al bloqueo de escritura de la publicación H1 de Fase 8.

2. Implementación y responsabilidades

Responsable	Módulo	Implementación identificada
Valentino	Infraestructura	App, navegación, cliente HTTP, errores y Docker/Nginx.
Lautaro	Vinos	Catálogo, filtros, paginación, detalle, alta y edición.
José	Ventas	Cliente/sala, carrito, validaciones y comprobante de API.
Antonio	Transferencias	Alta, renglones, consulta, terminar y procesar.
Fabricio	Reportes	Ranking, ventas por sala y stock faltante con filtros.

3. Integración con FastAPI y PostgreSQL

Los formularios trabajan con identificadores provenientes de los catálogos de la API. En ventas, React no calcula el total final: presenta los importes del comprobante devuelto por FastAPI. Las transferencias muestran los estados retornados por el backend; los errores de RN-09, RN-11 y RN-12 se notifican sin reemplazar los controles SQL.

Para la ejecución integrada se añadió **compose.fase7.yml**, un tercer servicio frontend construido con Node y servido por Nginx. El proxy de Nginx envía **/api/*** al servicio FastAPI, mientras el fallback SPA conserva las rutas de React Router al recargar.

4. Alcance de las pantallas

Pantalla	Flujo desarrollado y fuente
Vinos	Catálogo filtrable, ficha y formularios de alta/edición; **front/src/pages/vinos/**.
Ventas	Comprobante multirrenglón y confirmación con importes de API; **pages/ventas/**.
Transferencias	Cambio de estado y consulta por número; **pages/transferencias/**.
Reportes	Ranking, gráfico de ventas y stock por sala; **pages/reportes/**.

FASE 7 • PRUEBAS Y EVIDENCIAS

5. Validaciones documentadas

La carpeta **fase7/evidencias/** conserva pruebas de integración de módulos y del entorno Docker. Se registraron casos diferenciados de venta válida, rechazo RN-09, modificación de stock, alta de vino con auditoría RN-13, tablero de reportes y contenedores integrados. Además, la descripción del PR #8 documenta pruebas de transferencias válidas y rechazos de RN-11/RN-12 con rollback.

6. Evidencia representativa localizada en GitHub

Se identificaron las siguientes referencias concretas. Se incluyen los **archivos originales enlazados**, no imágenes recreadas; el cotejo visual de cada PNG queda pendiente antes de incrustarlo en la versión definitiva.

Prueba	Referencia original	Qué debe cotejarse al seleccionar la captura
Venta válida	01_venta_valida_150004_confirmada_react.png	Confirmación visible en React y número de comprobante.
Rechazo RN-09	04_rechazo_stock_insuficiente_RN09_409.png	Código 409 y mensaje de stock; cruzar con stock conservado.
Vino / RN-13	02_total_751_y_auditoria_precio_RN13.png	Cantidad de vinos y registro real de auditoría de precio.
Reportes	tablero_ranking.png	Tabla visible, datos devueltos por API y legibilidad.
Transferencias	integracion_transferencias/	Etapas del flujo: alta, renglones, estados y mensajes.
Docker	integracion_docker/	Contenedores, HTTP y conexión del frontend.

7. Alcance y limitaciones

Las rutas y componentes se comprobaron en el código de **integracion/fase7**. Las pruebas de negocio citadas corresponden a ensayos previos en entornos controlados; no se afirma aquí que todas las operaciones estén habilitadas en el sitio público. En Fase 8 se publicó primero una variante H1 de **solo lectura** y se preparó después una Preview DEMO separada. Su validación cloud corresponde al capítulo siguiente.

8. Conclusión y fuentes

La Fase 7 entregó una interfaz modular para consultar y operar mediante la API, con separación entre presentación y reglas de negocio. La integración Docker y los ensayos por módulo aportan evidencia de los flujos principales, mientras que la publicación y seguridad en internet se documentarán en Fase 8.

Fuentes: front/src/, **front/CONTRATO_COMPARTIDO.md**, **compose.fase7.yml**, **front/nginx.conf**, **fase7/evidencias/** y PR #8. Capturas seleccionadas: rutas exactas en el repositorio y enlaces activos en este PDF.

INFORME FASE 8

La Fase 8 llevó la solución de tres niveles a internet. Su objetivo fue ofrecer una web accesible por HTTPS, API conectada a PostgreSQL en la nube y pruebas de reglas de negocio sin comprometer el dataset principal. La validación funcional del despliegue fue completada y el equipo decidió dar por cerrado el trabajo técnico, manteniendo las observaciones documentales identificadas.

1. Objetivo y alcance

Se desplegaron **dos entornos independientes**: uno principal con datos FULL y consultas de solo lectura y otro institucional DEMO, autenticado, que habilita altas, ediciones, ventas, ingresos, transferencias y reportes sobre datos ficticios. La separación preserva la base principal mientras la cátedra utiliza los formularios de evaluación.

2. Arquitectura y despliegue

Nivel	Entorno FULL / consulta	Entorno DEMO / operación
Frontend React / Vercel	bodegas-del-atuel.vercel.app	bodegas-del-atuel-demo-profesores.vercel.app
API FastAPI / Render	bodegas-atuel-api.onrender.com	bodegas-atuel-demo-api.onrender.com
Base / Supabase	Proyecto sjfyocokehaubuwwbhnw	Proyecto zfxocudcvriicrzgmxie
Acceso	Visitantes: lectura H1	Profesores: inicio de sesión y escritura autorizada

La web institucional se publicó en un proyecto Vercel independiente, rama **fase8-vercel-preview-demo-20261110**, commit **9339932a1abe661cc1d026376ae1db72db7036be**. El archivo **front/vercel.json** de esa rama reenvía **/api/*** exclusivamente a la API DEMO. La publicación fue verificada desde navegador por Lautaro; coordinación también informó acceso correcto.

3. Proveedores evaluados

Se eligieron **Vercel Hobby** para el frontend, **Render Free** para FastAPI y **Supabase Free** para PostgreSQL. Frente a la alternativa Render PostgreSQL Free, Supabase ofrecía una vía más adecuada para conservar una base académica: Render documentaba vencimiento de bases gratuitas tras 30 días. Se consideró además Railway, pero sus recursos gratuitos limitados no aportaban la misma previsibilidad para mantener todo el sistema en coste cero. Los límites de planes gratuitos deben revisarse antes de nuevas altas de servicios.

Referencias comerciales utilizadas: *render.com/docs/free, supabase.com/pricing, vercel.com/docs/plans/hobby y docs.railway.com/pricing (consulta documental de noviembre de 2026).*

4. Migraciones, roles y tamaño de base

El esquema procede de **db/migrations/001_esquema.sql** a **008_indices.sql**. PostgreSQL 17.11 se utilizó en la nube y PostgreSQL 18 en el entorno local. El entorno FULL conserva **27 tablas**, **20.000 clientes**, **150.000 comprobantes** y **500.000 renglones de comprobante**, comprobados mediante consultas de solo lectura; satisface ampliamente el mínimo de 10.000 registros exigido en una tabla.

5. Seguridad y separación de entornos

La conexión a la base usa **TLS verify-full** y secretos cargados en el proveedor. La restricción de red de DEMO fue contrastada con un rechazo real por IP no autorizada (**EADDRNOTALLOWED**); una IP administrativa temporal fue retirada. El rol de API no es propietario ni superusuario; en DEMO puede ejecutar los procedimientos de ingresos y transferencias, pero no tiene UPDATE general sobre stock_sala.

El acceso de evaluadores usa cookies **HttpOnly, Secure, SameSite=Lax**, sesiones firmadas y protección CSRF. Se probaron inicio de sesión, recarga F5 y cierre. La limitación de intentos en memoria es un control complementario, no protección integral frente a tráfico masivo. Las credenciales no se incluyen en el repositorio ni en el informe.

6. Integración y pruebas funcionales — Bloque 4

Lautaro ejecutó siete escenarios desde la Preview DEMO y luego comprobó navegación y lectura en la URL estable. Las escrituras se contrastaron mediante consultas SQL independientes sobre Supabase DEMO. Los resultados se muestran en la tabla siguiente; no se repitieron operaciones sobre FULL.

Caso	Resultado verificado
Vinos / RN-13	Vino 4, cepa 1 al 100 %, maridaje 1 (16 °C); precio $2.000 a $2.200, auditoría registrada por bodega_api.
Clientes	Cliente ficticio 3 creado y editado; dirección persistida.
Venta válida	Comprobante 5, vino 1 x1, subtotal $1.300, descuento $130, total $1.170; stock sala 1: 30 a 29.
RN-09	Rechazo HTTP 409/P0001 informado por navegador al pedir 31 unidades con 30 disponibles; stock del vino 2 en sala 1 permanece en 30.
Ingreso	Ingreso 1 procesado, transportista 1, vino 2 x2; depósito: 120 a 122.
Transferencia	Solicitud 3 procesada, vino 3 x1 a sala 2; depósito: 120 a 119; sala: 25 a 26.
Reportes	Cinco consultas web correctas: ranking, ventas por sala, sin stock, transportista activo y trazabilidad.

La diferencia observada en el ranking y las ventas por sala fue explicada por los comprobantes **3 y 5**, ambos por $1.170: el acumulado de sala 1 es **$2.340** y el vino 1 suma **dos botellas**. También se verificaron cero comprobantes sin renglones y cero cantidades negativas de stock.

7. Respaldo y restauración del entorno principal

El respaldo FULL se realizó con **pg_dump** en formato CUSTOM, archivo **g1c_final_full_REINTENTO_20261109_225055.dump**, tamaño **10.452.417 bytes**. SHA-256 comunicado mediante Get-FileHash: **B5E48E30A0AB0011E99DD2FA3664C0C315B15BC5F4C7196EA3106A6571BED055**. Se inspeccionó la estructura del archivo mediante **pg_restore --list**.

El equipo restauró el respaldo en un PostgreSQL **17.11 local aislado** y comparó las consultas contra FULL: **27/27 conteos de tablas** coincidentes y **15/15 controles estructurales numéricos** coincidentes. Se constataron 32 claves foráneas válidas, 11 triggers de usuario activos y stock no negativo. La restauración se considera funcional para el alcance técnico probado.

Observación: no se encontró el log primario del comando pg_restore ni su código de salida original. No se acreditó identidad fila por fila ni equivalencia de propietarios/ACL, especialmente porque se usó --no-owner --no-acl. Estas limitaciones no se presentan como pruebas inexistentes. No se exige aquí un backup DEMO adicional para acreditar el respaldo FULL; el intento de backup DEMO no quedó validado y constituye un riesgo operativo independiente.

8. Evidencias, reproducción y cierre

Las actas y consultas están versionadas en **fase8/ACTA_CONTROL_FINAL_2026-11-10.md**, **fase8/CIERRE_Y_EVIDENCIAS.md** y **fase8/ACTA_ANEXO_DEMO_PUBLICADA_2026-11-10.md**. El PR documental **#23** se fusionó a **integracion/fase8** en el commit **c1e5488a2b9263a113ecd32465c6e4de076b8264**; se trató de documentación y evidencias, sin integración de código DEMO en FULL.

Se da por **cerrada técnicamente la Fase 8** por decisión de coordinación, con aplicaciones accesibles, reglas probadas y copia FULL restaurada y comprobada mediante los controles detallados. La validación formal por la cátedra y las limitaciones probatorias del log original se consignan por separado.

CIERRE DOCUMENTAL · FASE 9

La Fase 9 no añade un informe independiente. Su finalidad documental es **completar este informe integrador** y facilitar a un tercero los pasos para reproducir el proyecto, las migraciones, el dataset, la API, el frontend y las pruebas. La demostración y defensa oral corresponden al día de presentación y no constituyen archivos que se incorporen aquí.

1. README y reproducción

El README principal se encuentra en el repositorio y fue actualizado con la URL institucional de evaluación. Describe requisitos locales, ejecución en PowerShell, Docker Compose, migraciones versionadas, carga, pruebas y accesos HTTPS. Las escrituras de evaluación deben apuntar a DEMO, no a FULL.

Repositorio: **https://github.com/videlalauti/trabajo_integrador_bd**. Rama integrada: **integracion/fase8**. Publicación de evaluación: **https://bodegas-del-atuel-demo-profesores.vercel.app**. Contraseña entregada por canal privado.

2. Tabla de trazabilidad RN-01 a RN-15

La tabla de trazabilidad ya forma parte del **informe original de Fase 2** página 19 , preservado dentro de este mismo PDF. La implementación y las pruebas ampliadas se encuentran en las migraciones SQL, el conjunto de pruebas API y las actas de Fases 7 y 8. No se reemplaza esa tabla histórica por otra sin aprobación del equipo; para la entrega puede acompañarse de la matriz detallada como anexo externo si se requiere.

3. Documentos y evidencias conservadas

Archivo o ubicación	Contenido
docs/fase1.pdf	Documentación original del modelado físico.
Informes de Fase 2, 3 y 4	Reproducidos íntegramente en las páginas históricas de este PDF.
consultas/	Sentencias SQL comentadas de Fase 5.
api/, api/tests/	FastAPI, controles, pruebas automatizadas de Fase 6.
front/, fase7/evidencias/	Aplicación React y evidencias de Fase 7.
fase8/evidencias/	Conteos FULL de origen/destino, inventario estructural, rechazo IP, Gitleaks y actas.
README.md	Instrucciones de ejecución y URLs de las dos publicaciones.

4. Conclusión general

A lo largo del trabajo transformamos el diseño de una base de datos en un sistema que hoy puede consultarse y utilizarse desde la web. Cada fase nos permitió aplicar lo aprendido en clase a situaciones concretas: definir las relaciones, mantener la integridad, cargar grandes volúmenes de datos y asegurar que las operaciones cumplieran las reglas del negocio.

Lo más valioso fue entender que las partes del sistema tenían que funcionar juntas. Las ventas, los ingresos y las transferencias debían actualizar correctamente el stock; los errores tenían que rechazarse sin dejar datos inconsistentes. También aprendimos a medir consultas, organizar tareas entre compañeros y resolver problemas reales al conectar PostgreSQL, la API y la interfaz.

Finalmente, logramos publicar un entorno principal protegido y otro de demostración en el que se pueden probar las operaciones con datos ficticios. Verificamos el respaldo principal mediante una restauración y controles de integridad, dejando documentadas las limitaciones de la evidencia disponible. Más allá del resultado técnico, este proyecto nos permitió recorrer todas las etapas de desarrollo y reconocer la importancia de probar, revisar y documentar cada decisión.